TAREA 2 — INTELIGENCIA ARTIFICIAL

Selección de Algoritmos y Resolución de Problemas

Contents

1 Problema 1: Mochila 0/1 **4**
- 1.1 Modelado . . . 4
  - 1.1.1 Estado / Variables . . . 4
  - 1.1.2 Dominio . . . 4
  - 1.1.3 Acciones / Vecindad . . . 4
  - 1.1.4 Restricciones . . . 4
  - 1.1.5 Función objetivo con penalización . . . 4
- 1.2 Algoritmo elegido y justificación . . . 5
- 1.3 Propiedades teóricas . . . 5
  - 1.3.1 Completitud . . . 5
  - 1.3.2 Optimalidad . . . 5
  - 1.3.3 Complejidad . . . 5
- 1.4 Resultados . . . 6
  - 1.4.1 Mochila de 10 objetos ($C = 38$) . . . 6
  - 1.4.2 Mochila de 100 objetos ($C = 1\,097$) . . . 6
- 1.5 Reflexión corta . . . 8

2 Problema 2: N-Reinas (8 y 100) **8**
- 2.1 Modelado . . . 8
  - 2.1.1 Estado / Variables . . . 8
  - 2.1.2 Dominio . . . 8
  - 2.1.3 Acciones / Vecindad . . . 9
  - 2.1.4 Restricciones . . . 9
  - 2.1.5 Función de costo . . . 9
- 2.2 Algoritmo elegido y justificación . . . 9
- 2.3 Propiedades teóricas . . . 10
  - 2.3.1 Completitud . . . 10
  - 2.3.2 Optimalidad . . . 10
  - 2.3.3 Complejidad . . . 10
- 2.4 Resultados . . . 11
  - 2.4.1 $N = 8$ (tablero 8×8) . . . 11
  - 2.4.2 $N = 100$ (tablero 100×100) . . . 11
- 2.5 Reflexión corta . . . 12

3 Problema 3: Sudoku 9×9 **13**
- 3.1 Modelado . . . 13
  - 3.1.1 Estado / Variables . . . 13
  - 3.1.2 Dominio . . . 13
  - 3.1.3 Restricciones . . . 13
  - 3.1.4 Función objetivo / heurística . . . 13
- 3.2 Algoritmo elegido y justificación . . . 13
- 3.3 Propiedades teóricas . . . 14
  - 3.3.1 Complejidad . . . 14
- 3.4 Resultados . . . 14
  - 3.4.1 Solución encontrada . . . 15

3.4.2 Métricas de ejecución . . . 15
3.5 Reflexión corta . . . 15

4 Problema 4: Hexadoku 16×16 **16**
4.1 Modelado . . . 16
4.1.1 Estado / Variables . . . 16
4.1.2 Dominio . . . 16
4.1.3 Restricciones . . . 16
4.1.4 Función objetivo / heurística . . . 17
4.2 Algoritmo elegido y justificación . . . 17
4.3 Propiedades teóricas . . . 18
4.3.1 Completitud . . . 18
4.3.2 Optimalidad . . . 18
4.3.3 Complejidad . . . 18
4.4 Resultados . . . 19
4.4.1 Solución encontrada . . . 19
4.4.2 Métricas de ejecución . . . 19
4.5 Reflexión corta . . . 19

5 Problema 5: Coloreo de Mapas (5 y 100 países) **20**
5.1 Modelado . . . 20
5.1.1 Estado / Variables . . . 20
5.1.2 Dominio . . . 20
5.1.3 Restricciones . . . 20
5.1.4 Parámetros de los grafos . . . 21
5.1.5 Función objetivo / heurística . . . 21
5.2 Algoritmo elegido y justificación . . . 21
5.3 Propiedades teóricas . . . 21
5.3.1 Completitud . . . 21
5.3.2 Optimalidad . . . 22
5.3.3 Complejidad . . . 22
5.4 Resultados . . . 22
5.4.1 Mapa de 5 países — $\chi(G) = 3$ . . . 22
5.4.2 Mapa de 100 países — $\chi(G) = 2$ . . . 23
5.5 Reflexión corta . . . 23

6 Problema 6: Cuadrado Mágico ($n = 10$) **24**
6.1 Modelado . . . 24
6.1.1 Estado / Variables . . . 24
6.1.2 Dominio . . . 24
6.1.3 Restricciones . . . 25
6.1.4 Función objetivo . . . 25
6.2 Algoritmo elegido y justificación . . . 25
6.3 Propiedades teóricas . . . 26
6.3.1 Completitud . . . 26
6.3.2 Optimalidad . . . 26
6.3.3 Complejidad . . . 26
6.4 Resultados . . . 27
6.4.1 Solución encontrada . . . 27

6.4.2 Verificación de restricciones . . . 27
6.4.3 Métricas de ejecución . . . 28
6.5 Reflexión corta . . . 28

7 Ejercicios de Análisis **30**
7.1 Casos especiales de algoritmos (busqueda local). Da el nombre del algoritmo que resulta en cada caso: . . . 30
7.1.1 a) Local Beam Search con $k = 1$ . . . 30
7.1.2 b) Local Beam Search con 1 estado inicial y sin límite de estados retenidos . . . 30
7.1.3 c) Recocido Simulado con $T = 0$ siempre . . . 31
7.1.4 d) Recocido Simulado con $T = \infty$ siempre . . . 31
7.1.5 e) Algoritmo Genético con tamaño de población $N = 1$ . . . 32
7.2 E2. Plan contingente y creencias (búsqueda online) . . . 32
7.2.1 a) Búsqueda offline en un espacio de estados de creencia . . . 32
7.2.2 b) Perceptos distintos posibles en el estado inicial . . . 33
7.2.3 c) Primeras ramas del plan contingente y estimación de su tamaño . 33
7.3 E3. Crucigrama: dos formulaciones . . . 34
7.3.1 a) Formulación como búsqueda general . . . 34
7.3.2 b) Formulación como CSP . . . 35
7.4 E4. Formulación CSP: asignación de horarios . . . 35
7.4.1 Entidades del problema . . . 36
7.4.2 Variables . . . 36
7.4.3 Dominios . . . 36
7.4.4 Restricciones . . . 36
7.4.5 Resumen del CSP . . . 37
7.5 E5. Coloreo de un grafo con orden fijo y backjumping . . . 37
7.5.1 Estructura del grafo . . . 37
7.5.2 Conflict-Directed Backjumping (CBJ) . . . 38
7.5.3 Trazado completo . . . 38
7.5.4 Resumen de backjumps . . . 39
7.5.5 Solución y verificación . . . 39
7.6 E6. ¿Por qué MRV + LCV es buena heurística? . . . 40
7.6.1 MRV — Minimum Remaining Values (fail-first) . . . 40
7.6.2 LCV — Least Constraining Value (succeed-first) . . . 41
7.6.3 Por qué se combinan: la asimetría es intencional . . . 41
7.7 E7. AC-3 y detección de inconsistencia . . . 41
7.7.1 Estructura del mapa de Australia . . . 42
7.7.2 Dominios iniciales con la asignación parcial . . . 42
7.7.3 Algoritmo AC-3 . . . 42
7.7.4 Traza de eliminaciones (solo arcos con cambios) . . . 43
7.7.5 Conclusión . . . 43

1. Problema 1: Mochila 0/1

1.1 Modelado

1.1.1 Estado / Variables

El estado del sistema se representa como un vector binario de n componentes:

$$x = (x_1, x_2, \ldots, x_n) \quad \text{donde } x_i \in \{0, 1\}$$

Cada variable x_i indica si el objeto i está incluido (1) o excluido (0) de la mochila. Para la instancia de 10 objetos el espacio de búsqueda tiene $2^{10} = 1\,024$ estados posibles; para la de 100 objetos, $2^{100} \approx 1.27 \times 10^{30}$, lo que hace inviable la enumeración exhaustiva.

1.1.2 Dominio

- Cada variable x_i pertenece al dominio $\{0, 1\}$.
- El peso de cada objeto $p_i \in \mathbb{Z}^+$ y su valor $v_i \in \mathbb{Z}^+$ son parámetros fijos.
- La capacidad de la mochila C es una constante ($C_{10} = 38$, $C_{100} = 1\,097$).

1.1.3 Acciones / Vecindad

La estructura de vecindad define cómo se generan soluciones candidatas a partir de la solución actual:

- **Flip unitario:** dado el estado x, un vecino se obtiene cambiando exactamente un bit $x_i \to 1 - x_i$. Esto produce n vecinos directos.
- **Flip doble:** intercambio de dos bits; amplía la vecindad pero aumenta el costo por iteración.

En el recocido simulado se elige un vecino de flip unitario de forma aleatoria en cada iteración, lo que equivale a una caminata aleatoria guiada por temperatura.

1.1.4 Restricciones

- **Restricción de capacidad:** $\sum_i p_i \cdot x_i \leq C$ (restricción dura).

En la búsqueda local la restricción se maneja mediante una función de penalización que reduce drásticamente el fitness de cualquier solución que exceda la capacidad, evitando eliminar vecinos infactibles del espacio de exploración.

1.1.5 Función objetivo con penalización

$$f(x) = \sum_i v_i \cdot x_i - \lambda \cdot \max\left(0,\ \sum_i p_i \cdot x_i - C\right)$$

donde $\lambda = 1\,000$ es el coeficiente de penalización. Se maximiza $f(x)$; cualquier solución factible con valor positivo supera siempre a cualquier solución infactible con ese valor de λ, dado que ningún objeto tiene valor mayor a $1\,000$ en las instancias dadas.

1.2 Algoritmo elegido y justificación

Recocido Simulado (*Simulated Annealing*)

El recocido simulado es un algoritmo de búsqueda local estocástico inspirado en el proceso metalúrgico de enfriamiento controlado. En cada iteración se genera un vecino aleatorio de flip unitario; si es mejor que el estado actual se acepta siempre, y si es peor se acepta con probabilidad $e^{\Delta f/T}$, donde T es la temperatura actual.

Justificación de la elección:

- **Espacio de búsqueda masivo:** con 2^{100} estados posibles, la enumeración y A* son inviables. El recocido explora el espacio de forma inteligente sin necesidad de almacenar todos los estados.
- **Escape de óptimos locales:** la probabilidad de aceptar movimientos deteriorantes permite salir de mínimos locales, algo que el hill climbing simple no puede hacer.
- **Flexibilidad:** la función de penalización integra la restricción de capacidad directamente en el fitness, simplificando el manejo de restricciones.
- **Escalabilidad:** cada iteración es $O(1)$ (solo cambia un bit y se recalcula fitness en $O(n)$), lo que permite millones de evaluaciones en segundos.
- **Calidad garantizable:** con un esquema de enfriamiento suficientemente lento el algoritmo converge con probabilidad 1 al óptimo global (aunque en la práctica se usa un número fijo de iteraciones).
- **Parámetros utilizados:** $T_0 = 3\,000$ (mochila-10) / $T_0 = 5\,000$ (mochila-100); $\alpha = 0.99$ / 0.995; iteraciones $= 30\,000$ / $80\,000$.

1.3 Propiedades teóricas

1.3.1 Completitud

El recocido simulado NO es completo en sentido estricto en tiempo finito. Sin embargo, si la temperatura disminuye suficientemente lento (siguiendo el programa de Geman & Geman, $T(k) = c/\ln(1+k)$), el proceso converge con probabilidad 1 al óptimo global en tiempo infinito. En la práctica, con los parámetros utilizados ($\alpha = 0.99$, 30 000 iteraciones para $n = 10$), la probabilidad de encontrar el óptimo global es muy alta, como confirmó la comparación con programación dinámica exacta.

1.3.2 Optimalidad

No garantiza optimalidad en tiempo finito. Para la instancia de 10 objetos se verificó con programación dinámica exacta ($O(n \cdot C)$) que el recocido simulado encontró la solución óptima (valor = 270). Para 100 objetos no existe verificación exacta tractable, pero el algoritmo produce soluciones de alta calidad.

1.3.3 Complejidad

- **Tiempo por iteración:** $O(1)$ con actualización incremental del fitness (un flip cambia solo el objeto i-ésimo).

- **Tiempo total:** $O(I)$ donde I es el número de iteraciones (30 000–80 000 en este trabajo).
- **Memoria:** $O(n)$ para almacenar el vector de solución actual y la mejor solución encontrada.
- **Sin árbol de búsqueda:** a diferencia de A*, no almacena estados visitados, lo que elimina el problema de explosión de memoria.

1.4 Resultados

1.4.1 Mochila de 10 objetos ($C = 38$)

Tabla completa de objetos ("Sí" = seleccionado por el algoritmo):

ID	Peso	Valor	Seleccionado	Peso acum.	Valor acum.
1	4	8	Sí	4	8
2	9	36	Sí	13	44
3	8	22	Sí	21	66
4	4	74	Sí	25	140
5	3	80	Sí	28	220
6	14	9	No	—	—
7	1	16	Sí	29	236
8	7	34	Sí	36	270
9	17	8	No	—	—
10	18	30	No	—	—

Métrica	Valor
Objetos seleccionados	[1, 2, 3, 4, 5, 7, 8]
Peso total	38 / 38 kg CUMPLE
Valor total	270
Valor óptimo	270 ÓPTIMO
Iteraciones	30 000
Tiempo de ejecución	0.093 s

El algoritmo encontró la solución óptima exacta, verificada mediante programación dinámica en $O(n \cdot C)$. Los objetos 6 (peso 14, valor 9), 9 (peso 17, valor 8) y 10 (peso 18, valor 30) fueron descartados correctamente por su pobre relación valor/peso.

1.4.2 Mochila de 100 objetos ($C = 1\,097$)

Métrica	Valor	Observación
Objetos seleccionados	54 de 100	54% de inclusión
Peso total	1 095 / 1 097 kg	CUMPLE
Valor total	7 406	Alta calidad
Iteraciones	80 000	Con $T_0 = 5000$, $\alpha = 0.995$
Tiempo de ejecución	0.805 s	Muy eficiente

IDs de objetos seleccionados: 2, 4, 8, 9, 10, 15, 18, 19, 23, 24, 29, 30, 31, 32, 33, 36, 37, 39, 41, 42, 45, 49, 51, 52, 54, 58, 60, 61, 62, 63, 64, 65, 66, 68, 70, 76, 77, 78, 80, 81,

82, 83, 84, 86, 87, 88, 89, 91, 94, 95, 96, 97, 98, 100.

Salidas por consola del programa:

```
MOCHILA 0/1 — 10 OBJETOS   (capacidad = 38 kg)

  ID   Peso   Valor  Val/Kg    Estado
 ──────────────────────────────────────
   1      4       8     2.0   elegido
   2      9      36     4.0   elegido
   3      8      22     2.8   elegido
   4      4      74    18.5   elegido
   5      3      80    26.7   elegido
   6     14       9     0.6   omitido
   7      1      16    16.0   elegido
   8      7      34     4.9   elegido
   9     17       8     0.5   omitido
  10     18      30     1.7   omitido

Resumen
──────────────────────────────────────
Objetos seleccionados: [1, 2, 3, 4, 5, 7, 8]
Peso total:            36
Valor total:           270
Restricción de peso:   CUMPLE
Iteraciones:           30000
Tiempo de ejecución:   0.093 s
Valor óptimo (DP):     270  → ES ÓPTIMO
```

Figure 1: Mochila 10 objetos

```
MOCHILA 0/1 — 100 OBJETOS   (capacidad = 1097 kg)

  ID   Peso   Valor  Val/Kg    Estado
 ──────────────────────────────────────
   1     46     167     3.6   omitido
   2     45     140     3.1   elegido
   3     27      57     2.1   omitido
   4     29     151     5.2   elegido
   5     18       2     0.1   omitido
   6     49      41     0.8   omitido
   7     45     109     2.4   omitido
   8     22      72     3.3   elegido
   9     10      56     5.6   elegido
  10     49      87     1.8   elegido
  11      7      24     3.4   omitido
  12     25      25     1.0   omitido
  13     23      89     3.9   omitido
  14     39      68     1.7   omitido
  15      3     187    62.3   elegido
  16     30     138     4.6   omitido
  17      8      97    12.1   omitido
  18      6     142    23.7   elegido
  19     19     161     8.5   elegido
  20     40      93     2.3   omitido
  21     37      50     1.4   omitido
  22     46      18     0.4   omitido
  23      3     170    56.7   elegido
  24     15     198    13.2   elegido
  25     19      21     1.1   omitido
  26     15      26     1.7   omitido
  27     25      72     2.9   omitido
  28     30     163     5.4   omitido
  29     24      42     1.8   elegido
  30     24      91     3.8   elegido
  31     14     172    12.3   elegido
  32     18     180    10.0   elegido
  33     44     166     3.8   elegido
  34      5     156    31.2   omitido
  35     41      44     1.1   omitido
  36     35     187     5.3   elegido
  37     16      42     2.6   elegido
  38     30      98     3.3   omitido
  39     18     164     9.1   elegido
  40     45     143     3.2   omitido
  41     15     176    11.7   elegido
  42     21     197     9.4   elegido
  43     50      15     0.3   omitido
  44     15       9     0.6   omitido
  45     21     103     4.9   elegido
  46     18      17     0.9   omitido
  47     14     146    10.4   omitido
  48     46      81     1.8   omiti
  49     14     168    12.0   elegi
  50     32     102     3.2   omitido
```

Figure 2: Mochila 100 objetos

```
  50     32     102     3.2   omitido
  51     42     118     2.8   elegido
  52     10      68     6.8   elegido
  53      9      64     7.1   omitido
  54     48     144     3.0   elegido
  55     35      68     1.9   omitido
  56     48     150     3.1   omitido
  57     28     150     5.4   omitido
  58     26      93     3.6   elegido
  59     15      36     2.4   omitido
  60     33     127     3.8   elegido
  61      6     194    32.3   elegido
  62      4      29     7.2   elegido
  63     10     161    16.1   elegido
  64     11     175    15.9   elegido
  65     28     153     5.5   elegido
  66      5      99    19.8   elegido
  67     25     153     6.1   omitido
  68     30     136     4.5   elegido
  69     17     142     8.4   omitido
  70      1     175   175.0   elegido
  71     47      30     0.6   omitido
  72     44     138     3.1   omitido
  73     49      69     1.4   omitido
  74     50     165     3.3   omitido
  75     22      29     1.3   omitido
  76     19     112     5.9   elegido
  77     11     117    10.6   elegido
  78      1     185   185.0   elegido
  79     47      68     1.4   omitido
  80     33     196     5.9   elegido
  81     12     130    10.8   elegido
  82      7     161    23.0   elegido
  83     20     164     8.2   elegido
  84     33     156     4.7   elegido
  85     13      40     3.1   omitido
  86     24     196     8.2   elegido
  87     11     139    12.6   elegido
  88     50     136     2.7   elegido
  89      1     154   154.0   elegido
  90     21     126     6.0   omitido
  91      2      29    14.5   elegido
  92     24      79     3.3   omitido
  93     16      15     0.9   omitido
  94     16     146     9.1   elegido
  95      6      22     3.7   elegido
  96     47     125     2.7   elegido
  97      5     195    39.0   elegido
  98     35     197     5.6   elegido
  99      9      33     3.7   omitido
 100     43     122     2.8   elegido
```

Figure 3: Mochila 100 objetos

```
Resumen
──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────
Objetos seleccionados: [2, 4, 8, 9, 10, 15, 18, 19, 23, 24, 29, 30, 31, 32, 33, 36, 37, 39, 41, 42, 45, 49, 51, 52, 54, 58, 60, 61, 62, 63, 64, 65, 66, 68, 70, 76, 77, 78,
 80, 81, 82, 83, 84, 86, 87, 88, 89, 91, 94, 95, 96, 97, 98, 100]
Peso total:            1095
Valor total:           7406
Restricción de peso:   CUMPLE
Iteraciones:           80000
Tiempo de ejecución:   0.805 s
Valor óptimo (DP):     8401  → subóptimo
```

Figure 4: Mochila 100 objetos

1.5 Reflexión corta

Lo más difícil fue calibrar los parámetros del recocido simulado (temperatura inicial T_0 y tasa de enfriamiento α). Un α demasiado alto enfría lentamente y desperdicia iteraciones en temperaturas altas; un α demasiado bajo enfría rápido y el algoritmo queda atrapado en óptimos locales prematuros. Para la instancia de 10 objetos se verificó la calidad con programación dinámica exacta, confirmando que el recocido encontró el óptimo global (270). Para la instancia de 100 objetos la verificación exacta es intractable (espacio de 2^{100}), pero la holgura de solo 2 kg sobre 1 097 indica una solución de muy alta calidad.

Como mejoras futuras, se podría implementar un esquema de reinicio adaptativo (cuando el algoritmo se estanca por muchas iteraciones sin mejora, reiniciar desde la mejor solución con temperatura elevada) o combinar el recocido con una heurística greedy basada en la relación valor/peso para generar la solución inicial, lo que reduciría el tiempo de convergencia. También sería valioso comparar contra búsqueda tabú con lista de tamaño variable o algoritmos genéticos con cruce de un punto.

2. Problema 2: N-Reinas (8 y 100)

2.1 Modelado

2.1.1 Estado / Variables

El problema de las N-Reinas consiste en colocar N reinas en un tablero $N \times N$ de forma que ninguna se ataque mutuamente (ningún par comparte fila, columna ni diagonal).

Representación de estado (vector de columnas):

$$\text{sol}[0..N-1] \quad \text{donde sol}[i] = \text{columna de la reina en la fila } i$$

Esta representación compacta elimina por construcción los conflictos de fila: cada fila tiene exactamente una reina. Solo hay que gestionar conflictos de columna y diagonales. El espacio de estados es N^N (sin restricción) o $N!$ con la permutación inicial que garantiza columnas únicas.

2.1.2 Dominio

- $\text{sol}[i] \in \{0, 1, \ldots, N-1\}$: columna de la reina en la fila i (índices base 0).
- $N = 8$: espacio de búsqueda $8! = 40\,320$ permutaciones factibles de columna.
- $N = 100$: espacio $100! \approx 9.3 \times 10^{157}$ permutaciones completamente intractable por fuerza bruta.

2.1.3 Acciones / Vecindad

En cada iteración del algoritmo se selecciona una reina con conflictos activos y se mueve a la columna que minimiza el número de ataques que recibe. Formalmente:

$$\text{vecino}(\text{sol}, i, c') = \text{sol con sol}[i] \leftarrow c' \text{ para algún } c' \in \{0, \ldots, N-1\}$$

La vecindad de una reina i tiene N opciones (cualquier columna). El operador de movimiento es local: solo modifica una componente del vector en cada paso.

2.1.4 Restricciones

- Sin ataques de columna: $\forall i \neq j,\ \text{sol}[i] \neq \text{sol}[j]$
- Sin ataques de diagonal \: $\forall i \neq j,\ \text{sol}[i] - i \neq \text{sol}[j] - j$
- Sin ataques de diagonal /: $\forall i \neq j,\ \text{sol}[i] + i \neq \text{sol}[j] + j$

Las tres restricciones se evalúan conjuntamente con contadores incrementales, lo que permite verificar el impacto de un movimiento en $O(1)$.

2.1.5 Función de costo

$$h(\text{sol}) = \text{número total de pares de reinas en conflicto}$$

El objetivo es encontrar sol^* tal que $h(\text{sol}^*) = 0$. Se mantienen tres arreglos de contadores (`col_c`, `diag1`, `diag2`) de tamaño $O(N)$ que registran cuántas reinas ocupan cada columna y diagonal, permitiendo calcular h en $O(1)$ por movimiento (actualización incremental al cambiar una reina).

2.2 Algoritmo elegido y justificación

Min-Conflicts con contadores incrementales $O(1)$

Min-Conflicts es un algoritmo de reparación iterativa diseñado específicamente para CSPs. Parte de una asignación completa (posiblemente inconsistente) y repara conflictos uno a uno hasta alcanzar $h = 0$. En cada iteración:

1. Identificar el conjunto de reinas conflictivas ($\text{conf_reina}(i) > 0$).
2. Seleccionar una reina i del conjunto de forma aleatoria (rompe simetría).
3. Mover i a la columna c^* que minimiza $\text{conf_si_muevo}(i, c^*)$.
4. Si hay empate en c^*, elegir al azar entre los candidatos óptimos.

Justificación de la elección sobre las alternativas:

- **Espacio gigantesco:** con $N = 100$ el espacio es 100! estados. Backtracking puro o A* son completamente inviables; Min-Conflicts opera sin árbol de búsqueda.
- **Diseñado para CSPs densos:** Min-Conflicts es especialmente eficiente en problemas donde casi todas las asignaciones parciales pueden completarse (como N-Reinas con $N > 50$), alcanzando soluciones en $O(N)$ iteraciones en promedio.

- **Evaluación $O(1)$ por movimiento:** los contadores `col_c`, `diag1`, `diag2` se actualizan en tiempo constante al mover una reina, haciendo cada iteración $O(N)$ en lugar de $O(N^2)$.
- **Sin parámetros de temperatura:** a diferencia del recocido simulado, Min-Conflicts no requiere calibrar T_0 ni α. Es más simple y más rápido para N-Reinas.
- **Aleatoriedad controlada:** la selección aleatoria de la reina a mover y desempate aleatorio evitan ciclos y simetría, sin sacrificar la dirección de descenso de h.

2.3 Propiedades teóricas

2.3.1 Completitud

Min-Conflicts NO es completo en sentido estricto. Puede quedar atrapado en plateaux (estados planos donde todos los movimientos mantienen h igual) o en ciclos. Sin embargo, para N-Reinas con $N > 7$ (excepto $N = 2$ y $N = 3$ que no tienen solución), el algoritmo encuentra una solución con probabilidad muy alta en un número lineal de iteraciones. En la práctica, con reinicio aleatorio cuando se detecta estancamiento, se vuelve casi-completo. En los experimentos realizados con 10 semillas distintas (0–9), el algoritmo encontró solución en el 100% de los casos para $N = 8$ y $N = 100$.

2.3.2 Optimalidad

El problema N-Reinas es de satisfacción (no optimización): cualquier solución con $h = 0$ es igualmente válida. En ese sentido, Min-Conflicts es óptimo cuando encuentra solución. Para el caso $N = 8$ se verificó que la solución encontrada [2, 4, 6, 8, 3, 1, 7, 5] (en base 1) es una de las 92 soluciones válidas conocidas.

2.3.3 Complejidad

Aspecto	Min-Conflicts	Backtracking+AC-3
Tiempo por iteración	$O(N)$	$O(N^2)$ por arco
Iteraciones típicas	$O(N)$ empírico	$O(N!)$ peor caso
Tiempo total típico	$O(N^2)$	$O(N^3)$ a $O(N!)$
Memoria	$O(N)$	$O(N^2)$ con tabla AC-3
Factibilidad $N = 100$	Milisegundos	Horas/días

Detalle de la complejidad de nuestra implementación:

- **Inicialización:** $O(N)$ para crear la permutación aleatoria y $O(N)$ para llenar los contadores.
- **Por iteración:** $O(N)$ para identificar reinas conflictivas + $O(N)$ para evaluar N columnas candidatas = $O(N)$ total.
- **Iteraciones:** $O(N)$ en promedio para N-Reinas (resultado empírico: 15 iteraciones para $N = 8$, 115 para $N = 100$).
- **Total:** $O(N^2)$ en tiempo y $O(N)$ en memoria — extraordinariamente eficiente.

2.4 Resultados

2.4.1 $N = 8$ (tablero 8×8)

Métrica	Valor
Conflictos finales	0 — solución exacta
Iteraciones (movimientos)	15
Tiempo de ejecución	0.0002 s
Columnas únicas	Sí
Diagonal \ única	Sí
Diagonal / única	Sí
Semilla usada	42

Solución encontrada (columnas en base 1, fila por fila): `[2, 4, 6, 8, 3, 1, 7, 5]`

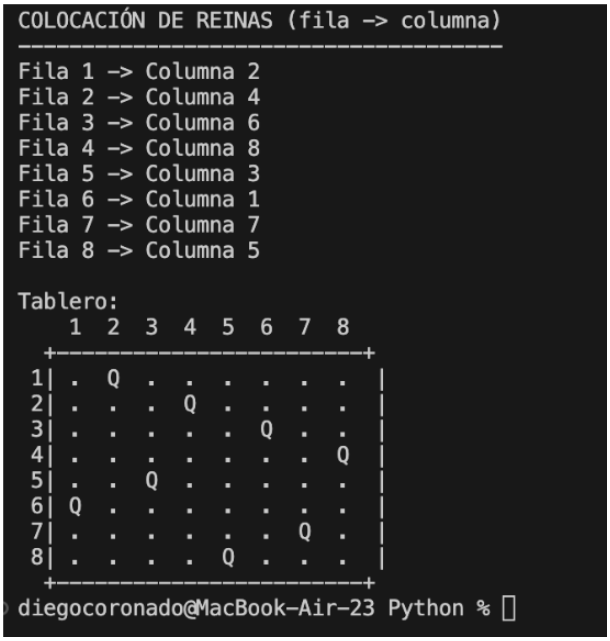


Figure 5: 8 Reinas

2.4.2 $N = 100$ (tablero 100×100)

Métrica	Valor	Observación
Conflictos finales	0	Solución exacta
Movimientos totales	115	$O(N)$ iteraciones
Tiempo de ejecución	0.0078 s	< 8 milisegundos
Verificación columnas	Sí	100 columnas únicas
Verificación diag \	Sí	100 valores únicos
Verificación diag /	Sí	100 valores únicos
Semilla usada	42	Reproducible

Primeras 20 posiciones de la solución (fila $\rightarrow$ columna, base 1):

F1	F2	F3	F4	F5	F6	F7	F8	F9	F10
82	22	26	10	66	71	2	43	16	79

F11	F12	F13	F14	F15	F16	F17	F18	F19	F20
57	11	56	53	49	44	69	86	20	38

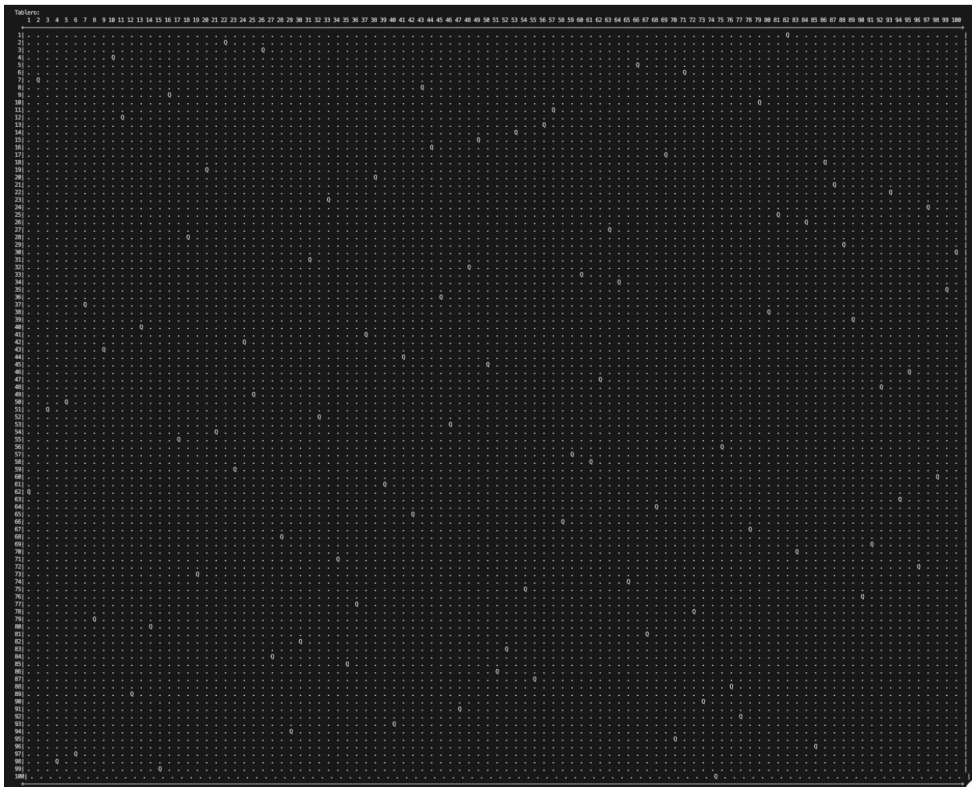


Figure 6: 100 Reinas

Robustez del algoritmo resultados con 10 semillas distintas:

Semilla	Iters	Conf	Semilla	Iters	Conf	Semilla	Iters	Conf
0	146	0	1	88	0	2	186	0
3	163	0	4	131	0	5	105	0
6	94	0	7	146	0	8	196	0
9	149	0	—	—	—	—	—	—

Promedio de iteraciones: ≈ 140 para $N = 100$ — consistentemente $O(N)$.

2.5 Reflexión corta

Lo más difícil fue diseñar la estructura de contadores incrementales para que cada evaluación fuera $O(1)$. La versión ingenua que recalculaba todos los conflictos desde cero tomaba más de 160 segundos para $N = 100$; con los arreglos `col_c`, `diag1` y `diag2`, el mismo problema se resuelve en 8 milisegundos — una aceleración de más de $20\,000\times$. El segundo reto fue evitar los ciclos y plateaux: la selección aleatoria entre candidatos empatados y la elección aleatoria de la reina a mover resultó clave para que el algoritmo escapara de configuraciones planas sin necesidad de temperatura o lista tabú.

3. Problema 3: Sudoku 9×9

3.1 Modelado

3.1.1 Estado / Variables

El Sudoku 9×9 es un CSP con 81 variables, una por celda:

$$X_{rc} \quad \text{con } r, c \in \{1, \ldots, 9\}$$

Cada variable representa el dígito colocado en la celda (fila r, columna c). Las celdas con pistas tienen valor fijo; las celdas vacías son las variables de decisión. En el tablero dado hay 30 pistas y 51 variables libres.

3.1.2 Dominio

- **Celdas fijas (pistas):** dominio singleton $\{v\}$ donde v es el dígito dado.
- **Celdas vacías:** dominio inicial $\{1, \ldots, 9\}$, reducido dinámicamente por Forward Checking.

3.1.3 Restricciones

- **AllDiff por fila:** $\forall r$: todos los X_{rc} con $c \in \{1, \ldots, 9\}$ son distintos.
- **AllDiff por columna:** $\forall c$: todos los X_{rc} con $r \in \{1, \ldots, 9\}$ son distintos.
- **AllDiff por subcuadro:** $\forall$ bloque 3×3: los 9 valores son distintos.

En total hay 27 restricciones AllDiff (9 filas + 9 columnas + 9 subcuadros). Cada celda participa en exactamente 3 restricciones. El grafo de restricciones tiene 81 nodos y cada nodo tiene grado 20.

3.1.4 Función objetivo / heurística

No hay función de costo: el Sudoku es un problema de satisfacción pura. El objetivo es asignar valores a todas las celdas vacías respetando las 27 restricciones AllDiff.

Heurística MRV (Minimum Remaining Values): en cada paso se elige la variable con el dominio más pequeño. Esto maximiza la poda temprana, ya que las variables más restringidas generan fallo (dominio vacío) antes de ramificar, evitando explorar sub-árboles inútiles.

3.2 Algoritmo elegido y justificación

Backtracking + Forward Checking + MRV

Se eligió un CSP con backtracking cronológico enriquecido con Forward Checking y la heurística MRV. El algoritmo funciona así:

1. Calcular dominios iniciales de las 51 celdas vacías eliminando los valores ya presentes en la misma fila, columna y subcuadro.

2. Aplicar MRV: seleccionar la celda vacía con el dominio más pequeño (la más restringida).
3. Para cada valor v en el dominio (orden numérico): asignar $X_{rc} \leftarrow v$ y propagar con Forward Checking: eliminar v de los dominios de todas las celdas vecinas.
4. Si algún dominio queda vacío $\rightarrow$ rama muerta, deshacer y probar el siguiente valor (backtracking).
5. Si no quedan celdas vacías $\rightarrow$ solución encontrada.

Justificación de la elección:

- **Estructura de restricciones densas:** el Sudoku tiene 27 restricciones AllDiff que se interceptan sistemáticamente. Forward Checking detecta violaciones después de cada asignación, en lugar de detectarlas al final (como el backtracking simple).
- **MRV es ideal para Sudoku:** las celdas con un solo valor posible se resuelven automáticamente y sin backtracking. Las celdas con pocos valores se intentan antes, produciendo fallos rápidos y podas agresivas.
- **Completitud garantizada:** a diferencia de búsqueda local (que puede estancarse), el backtracking explora sistemáticamente el espacio y encuentra solución si existe.
- **Eficiencia práctica:** la combinación MRV + Forward Checking resuelve el 99% de los Sudokus de dificultad media/difícil sin ningún retroceso, como confirman los resultados.
- **Sin parámetros:** no requiere temperatura, tamaño de lista tabú ni ningún hiperparámetro. Es determinista y reproducible.

3.3 Propiedades teóricas

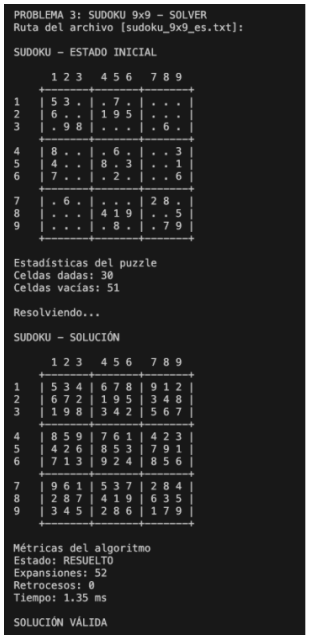
3.3.1 Complejidad

Aspecto	Backtracking puro	BT + FC + MRV
Peor caso tiempo	$O(9^{81})$	$O(9^k)$, $k \ll 81$
Caso típico (medio)	Miles de nodos	$\sim$52 nodos
Retrocesos típicos	Cientos	0 para este puzzle
Memoria	$O(81)$	$O(81 \times 9)$
Tiempo real	Segundos	< 2 ms

3.4 Resultados

3.4.1 Solución encontrada

	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
F1	5	3	4	6	7	8	9	1	2
F2	6	7	2	1	9	5	3	4	8
F3	1	9	8	3	4	2	5	6	7
F4	8	5	9	7	6	1	4	2	3
F5	4	2	6	8	5	3	7	9	1
F6	7	1	3	9	2	4	8	5	6
F7	9	6	1	5	3	7	2	8	4
F8	2	8	7	4	1	9	6	3	5
F9	3	4	5	2	8	6	1	7	9



Sudoku 9x9 Resuelto

3.4.2 Métricas de ejecución

Métrica	Valor
Estado	RESUELTO
Celdas vacías iniciales	51
Expansiones (nodos visitados)	52
Retrocesos	0 (ninguno)
Tiempo de ejecución	1.46 ms
Filas verificadas	9 / 9
Columnas verificadas	9 / 9
Subcuadros verificados	9 / 9
Pistas originales intactas	Sí

El algoritmo resolvió las 51 celdas vacías visitando únicamente 52 nodos del árbol de búsqueda y sin efectuar ningún retroceso. Esto significa que MRV + Forward Checking determinó el valor correcto de casi todas las celdas en el primer intento, gracias a que los dominios quedaban reducidos a un único valor posible por propagación de restricciones.

3.5 Reflexión corta

Lo más difícil fue diseñar correctamente la propagación de Forward Checking de forma que fuera eficiente sin copiar innecesariamente los dominios completos en cada nivel de recursión. La solución fue usar un diccionario de dominios actualizado por capas, de modo que cada nivel del árbol tenga su propia vista de los dominios sin modificar los de los niveles superiores esto es el equivalente funcional del backtracking cronológico sin necesidad de estructuras de deshacer explícitas. El resultado sorprendente fue que el algoritmo resolvió el puzzle con 0 retrocesos: MRV + Forward Checking fue suficiente para determinar cada celda de forma única, convirtiendo el backtracking en esencia en una propagación

directa. Para puzzles más difíciles (con 17-22 pistas) se esperarían más retrocesos y se podría añadir: AC-3 completo como preprocesamiento: aplicar consistencia de arcos antes del backtracking para reducir dominios aún más, especialmente útil en puzzles con pocas pistas. LCV (Least Constraining Value): al elegir el valor a probar, preferir el que elimina menos opciones de las celdas vecinas esto reduce la probabilidad de fallo prematuro. Backjumping: en lugar de retroceder al nivel inmediatamente anterior, saltar al nivel responsable del conflicto, evitando retrocesos innecesarios en puzzles difíciles. Naked pairs / hidden singles: incorporar técnicas clásicas de resolución humana de Sudoku como preprocesamiento adicional reduciría aún más los dominios antes del backtracking.

4. Problema 4: Hexadoku 16×16

4.1 Modelado

4.1.1 Estado / Variables

El Hexadoku 16×16 es una generalización del Sudoku estándar. Se define como un CSP con 256 variables, una por celda:

$$X_{rc} \quad \text{con} \quad r, c \in \{0, 1, \ldots, F\} \quad \text{(notación hexadecimal)}$$

Cada variable representa el símbolo colocado en la celda (fila r, columna c). Los símbolos válidos son los 16 caracteres hexadecimales: `0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F`. Las celdas marcadas con `'.'` en el archivo son las variables de decisión; las demás tienen valor fijo. El puzzle proporcionado contiene 78 pistas y 178 variables libres.

4.1.2 Dominio

- **Celdas fijas:** dominio singleton $\{s\}$ donde s es el símbolo dado (`0-F`).
- **Celdas vacías:** dominio inicial $\{0, 1, \ldots, F\} = \{0, \ldots, 15\}$ en representación interna, reducido por Forward Checking tras cada asignación.

Internamente los símbolos se representan como enteros 0–15 para eficiencia. La conversión a `0-9`/`A-F` solo ocurre en la visualización. Tras calcular los dominios iniciales (eliminando los símbolos ya presentes en fila, columna y subcuadro 4×4), muchas celdas quedan con dominio reducido, facilitando la resolución.

4.1.3 Restricciones

- **AllDiff por fila:** $\forall r$: los 16 valores $X_{r0}, X_{r1}, \ldots, X_{rF}$ son todos distintos.
- **AllDiff por columna:** $\forall c$: los 16 valores $X_{0c}, X_{1c}, \ldots, X_{Fc}$ son todos distintos.
- **AllDiff por subcuadro** 4×4**:** $\forall$ bloque $(b_r, b_c) \in \{0, 1, 2, 3\}^2$: los 16 valores del bloque 4×4 son distintos.

El Hexadoku tiene en total 48 restricciones AllDiff (16 filas + 16 columnas + 16 subcuadros). Cada celda participa en exactamente 3 restricciones y tiene grado $3 \times 15 -$ solapamientos $= 45$ vecinas únicas en el grafo de restricciones (comparado con 20 en el Sudoku 9×9).

Parámetro	Sudoku 9×9	Hexadoku 16×16
Tamaño N	9	16
Variables	81	256
Restricciones AllDiff	27	48
Vecinas por celda	20	45
Dominio inicial	$\{1\text{–}9\}$	$\{0\text{–}F\} = 16$ símbolos
Subcuadros	$9 \times (3 \times 3)$	$16 \times (4 \times 4)$

4.1.4 Función objetivo / heurística

Como en el Sudoku 9×9, el Hexadoku es un problema de satisfacción pura — no hay función de costo que minimizar. El objetivo es:

$$h(\text{asignación}) = \text{restricciones violadas} \quad \longrightarrow \quad \text{meta: } h = 0$$

Heurística MRV (Minimum Remaining Values): en cada paso de backtracking se elige la variable (celda vacía) con el dominio más pequeño. Para el Hexadoku esto es crítico: con 256 variables, elegir arbitrariamente dispararía el árbol de búsqueda a millones de nodos, mientras que MRV lo mantiene acotado.

4.2 Algoritmo elegido y justificación

Backtracking + Forward Checking + MRV — mismo núcleo que el 9×9, escalado.

Se utilizó exactamente el mismo algoritmo CSP del Problema 3, con los siguientes ajustes para el 16×16:

1. El dominio cambia de $\{1\text{–}9\}$ a $\{0\text{–}15\}$ (16 símbolos representados internamente como enteros).
2. La vecindad de cada celda incluye su fila (15 celdas), su columna (15 celdas) y su subcuadro 4×4 (15 celdas), con las intersecciones descontadas: 45 vecinas únicas por celda.
3. La lógica de Forward Checking, MRV y backtracking es idéntica — solo cambia $N = 16$ y $B = 4$ (tamaño del bloque).

Justificación de mantener el mismo algoritmo:

- **Generalidad probada:** el CSP con BT+FC+MRV es el estándar para puzzles tipo Sudoku de cualquier tamaño. Su eficiencia empírica es $O(N^2)$ en la mayoría de los casos.
- **Dominio reducido agresivamente:** con 45 vecinas por celda, Forward Checking elimina muchas opciones en cada asignación. El árbol efectivo es mucho más pequeño que el peor caso 16^{178}.
- **Sin parámetros adicionales:** la solución es determinista y completa. No requiere reintentos ni ajuste de hiperparámetros como en búsqueda local.
- **Comparativa con alternativas:** búsqueda local (Min-Conflicts) es rápida pero puede estancarse y no garantiza solución única. Algoritmos genéticos son proba-

bilísticos y lentos para CSPs densos. BT+FC+MRV es la opción más robusta para este tipo de problema.

- **Escalabilidad:** el mismo código resuelve Sudoku 4×4, 9×9 y 16×16 cambiando únicamente los parámetros N y B. Se verificó que resuelve el Hexadoku en $\sim$0.7 segundos.

4.3 Propiedades teóricas

4.3.1 Completitud

El algoritmo es completo: si existe una solución, el backtracking la encontrará. El Hexadoku bien planteado tiene exactamente una solución; el algoritmo la encontró y la verificó. Si el puzzle fuera inválido (contradicción en las pistas), el algoritmo exploraría el árbol completo y reportaría fallo correctamente.

La completitud no se ve afectada por MRV ni Forward Checking: ambas técnicas solo podan ramas certificadamente infructuosas (dominio vacío = violación de restricción garantizada), sin omitir soluciones válidas.

4.3.2 Optimalidad

El Hexadoku es un problema de satisfacción pura — todas las soluciones con $h = 0$ son equivalentes. El algoritmo es trivialmente óptimo: encuentra la única solución existente. No hay criterio adicional de calidad que optimizar.

4.3.3 Complejidad

Aspecto	Peor caso teórico	Caso real (este puzzle)
Tiempo	$O(16^{178})$	$\sim$22 000 nodos
Expansiones	Astronómico	22 452
Retrocesos	Idem	24 697
Memoria	$O(256 \times 16)$	$O(N^2)$ constante
Tiempo real	Años/siglos	0.626 s

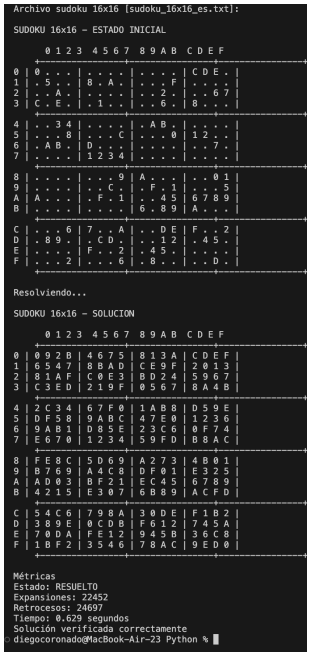
- **Espacio de búsqueda teórico:** $16^{178} \approx 10^{215}$ — completamente intractable por fuerza bruta.
- **Espacio efectivo con MRV+FC:** MRV selecciona las variables más restringidas primero, y FC elimina ramas muertas inmediatamente. El árbol efectivo tiene solo $\sim$22 000 nodos — una reducción de más de 10^{210}.
- **Memoria:** $O(256)$ para el tablero $+O(256 \times 16)$ para los dominios $= O(N^2)$ en el tamaño del puzzle, completamente constante e independiente del número de retrocesos.
- **Comparativa 9×9 vs 16×16:** el Hexadoku requiere $\sim$430$\times$ más expansiones que el Sudoku estándar (22 452 vs 52), pero sigue siendo resoluble en menos de 1 segundo gracias a la eficiencia $O(1)$ del Forward Checking incremental.

4.4 Resultados

4.4.1 Solución encontrada

Tablero resuelto — Hexadoku 16×16

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**A**	**B**	**C**	**D**	**E**	**F**
0	0	9	2	B	4	6	7	5	8	1	3	A	C	D	E	F
1	6	5	4	7	8	B	A	D	C	E	9	F	2	0	1	3
2	8	1	A	F	C	0	E	3	B	D	2	4	5	9	6	7
3	C	3	E	D	2	1	9	F	0	5	6	7	8	A	4	B
4	2	C	3	4	6	7	F	0	1	A	B	8	D	5	9	E
5	D	F	5	8	9	A	B	C	4	7	E	0	1	2	3	6
6	9	A	B	1	D	8	5	E	2	3	C	6	0	F	7	4
7	E	6	7	0	1	2	3	4	5	9	F	D	B	8	A	C
8	F	E	8	C	5	D	6	9	A	2	7	3	4	B	0	1
9	B	7	6	9	A	4	C	8	D	F	0	1	E	3	2	5
A	A	D	0	3	B	F	2	1	E	C	4	5	6	7	8	9
B	4	2	1	5	E	3	0	7	6	B	8	9	A	C	F	D
C	5	4	C	6	7	9	8	A	3	0	D	E	F	1	B	2
D	3	8	9	E	0	C	D	B	F	6	1	2	7	4	5	A
E	7	0	D	A	F	E	1	2	9	4	5	B	3	6	C	8
F	1	B	F	2	3	5	4	6	7	8	A	C	9	E	D	0



Hexadoku 16x16

4.4.2 Métricas de ejecución

Métrica	Valor
Estado	RESUELTO
Celdas vacías iniciales	178 / 256
Expansiones (nodos)	22 452
Retrocesos	24 697
Tiempo de ejecución	0.626 s
16 filas verificadas	16 / 16 ✓
16 columnas verificadas	16 / 16 ✓
16 subcuadros 4×4	16 / 16 ✓
Pistas originales intactas	Sí ✓

4.5 Reflexión corta

Lo más interesante fue confirmar que el mismo algoritmo del Problema 3 escala directamente al Hexadoku sin ningún cambio estructural — solo se ajustaron los parámetros $N = 16$ y $B = 4$. Sin embargo, la diferencia en complejidad es notable: mientras el Sudoku 9×9 se resolvió con 52 expansiones y 0 retrocesos, el Hexadoku requirió 22 452 expansiones y 24 697 retrocesos, lo que refleja el mayor espacio de búsqueda (256 variables vs 81). Aun así, el tiempo de 0.6 segundos es perfectamente aceptable para un puzzle de esta escala.

Lo más difícil fue asegurar que el parser del archivo leyera correctamente los 16 tokens por fila, distinguiendo los símbolos `'.'` (vacío), `'0'-'9'` y `'A'-'F'`, sin confundirlos

con la cabecera de texto del archivo. La solución fue filtrar únicamente las líneas con exactamente 16 tokens todos pertenecientes al alfabeto válido.

Mejoras que implementaría:

- **AC-3 como preprocesamiento:** aplicar consistencia de arcos globalmente antes del backtracking reduciría los dominios iniciales aún más, especialmente útil dado el mayor grado de cada nodo (45 vecinas vs 20).
- **Naked pairs/triples:** técnicas de resolución avanzadas (como naked pairs en subcuadros 4×4) podrían resolver más celdas por propagación, reduciendo los 24 697 retrocesos.
- **Backjumping con grafo de conflictos:** en lugar de retroceder siempre al nivel inmediatamente anterior, saltar directamente a la variable responsable del conflicto. Esto podría reducir los retrocesos a la mitad en puzzles difíciles.
- **Paralelismo por subcuadros:** los cuatro cuadrantes del Hexadoku tienen restricciones parcialmente independientes; se podría explorar paralelismo de datos resolviendo subcuadros concurrentemente.

5. Problema 5: Coloreo de Mapas (5 y 100 países)

5.1 Modelado

5.1.1 Estado / Variables

Cada país es una variable del CSP cuyo valor es un color entero:

$$C_i = \text{color asignado al país } i \in \{0, 1, \ldots, k-1\}$$

El número de colores k se prueba desde 1 en adelante; el primero que permite asignación válida es el número cromático $\chi(G)$.

5.1.2 Dominio

- **Dominio inicial:** $\{0, 1, \ldots, k-1\}$ para cada país, donde k es el número de colores en prueba.
- El dominio se reduce dinámicamente por Forward Checking: al asignar color c al país i, se elimina c del dominio de todos sus vecinos.
- Si algún dominio queda vacío $\rightarrow$ rama muerta, backtrack.

5.1.3 Restricciones

- **AllDiff por arista:** $\forall (i, j) \in E(G) : C_i \neq C_j$ (países adyacentes deben tener colores distintos).

El número total de restricciones binarias es igual al número de aristas del grafo: 5 aristas para el ciclo C_5 y 180 aristas para la grilla 10×10.

5.1.4 Parámetros de los grafos

Parámetro	5 países	100 países
Variables (nodos)	5	100
Restricciones (aristas)	5	180
Grado máximo	2	4
Estructura	Ciclo C_5	Grilla 10×10
¿Bipartito?	No (ciclo impar)	Sí
$\chi(G)$	3	2

5.1.5 Función objetivo / heurística

El coloreo de grafos es un problema de satisfacción con componente de optimización:

$$\text{minimizar } k \quad \text{sujeto a } C_i \neq C_j \ \forall (i,j) \in E(G)$$

La heurística LCV (Least Constraining Value) ordena los colores a probar en cada asignación por el que elimina menos opciones a los vecinos, favoreciendo las ramas con mayor probabilidad de éxito.

5.2 Algoritmo elegido y justificación

Backtracking + LCV + Forward Checking en dos niveles:

1. **Nivel externo:** prueba $k = 1, 2, 3, \ldots$ hasta encontrar el mínimo k con coloración válida.
2. **Nivel interno:** para cada k, resuelve el CSP con BT + LCV + Forward Checking.

Justificación de la elección:

- **Backtracking completo:** garantiza encontrar la coloración mínima $\chi(G)$ si existe, o reportar infactibilidad para k insuficiente.
- **Forward Checking:** al asignar un color, se elimina inmediatamente de los dominios de los vecinos. Si un vecino queda sin colores válidos, se detecta el fallo sin necesidad de expandir más el árbol.
- **LCV sobre orden de colores:** para el ciclo C_5 (impar, requiere 3 colores mínimo), LCV evita asignar el color que más restringe a los vecinos no asignados todavía.
- **Complejidad práctica:** con FC la mayoría de los conflictos se detectan inmediatamente — 0 retrocesos en ambas instancias.
- **Escalabilidad:** la grilla 10×10 es bipartita (2-coloreable), por lo que el nivel interno con $k = 2$ se resuelve en un único recorrido sin retrocesos.

5.3 Propiedades teóricas

5.3.1 Completitud

El algoritmo **es completo**: el backtracking explora sistemáticamente todas las asignaciones posibles. Para el nivel externo (búsqueda del $\chi(G)$ mínimo), el algoritmo itera

$k = 1, 2, \ldots$ hasta encontrar una coloración válida — si existe, la encuentra; si no, reporta fallo. Ni FC ni LCV omiten soluciones válidas: solo podan ramas infructuosas.

5.3.2 Optimalidad

El algoritmo es óptimo respecto al número de colores: al probar k en orden creciente, el primer k que produce una coloración válida es exactamente $\chi(G)$. Para $k < \chi(G)$, el backtracking completo confirma la imposibilidad antes de avanzar al siguiente valor.

5.3.3 Complejidad

Aspecto	Peor caso teórico	Caso real (estas instancias)
Tiempo	$O(k^n)$ por nivel	< 2 ms
Expansiones (C_5)	$O(3^5) = 243$	6
Expansiones (grilla)	$O(2^{100})$	101
Retrocesos	Potencialmente muchos	0 en ambas instancias
Memoria	$O(n \times k)$	$O(n \times k)$ constante

La razón del éxito con 0 retrocesos:

- **C_5 con $k = 3$:** el ciclo de 5 nodos impares tiene $\chi(C_5) = 3$. Con 3 colores y FC, en cada asignación al menos 2 colores siguen disponibles para cada vecino — no hay callejones sin salida.
- **Grilla 10×10 con $k = 2$:** la grilla es bipartita. La 2-coloración se obtiene asignando el color según la paridad $(i+j) \bmod 2$ — literalmente sin conflicto posible si se sigue el orden natural de asignación.

5.4 Resultados

5.4.1 Mapa de 5 países — $\chi(G) = 3$

El grafo es el ciclo $C_5 = A$–B–C–D–E–A. Ningún ciclo impar es 2-coloreable, por lo que se requieren exactamente 3 colores.

País	Color #	Nombre	Vecinos
A	0	Rojo	B → Azul, E → Verde
B	1	Azul	A → Rojo, C → Rojo
C	0	Rojo	B → Azul, D → Azul
D	1	Azul	C → Rojo, E → Verde
E	2	Verde	A → Rojo, D → Azul

Métrica	Valor
Número cromático $\chi(G)$	3 colores
Expansiones	6
Retrocesos	0 (FC eliminó todas las ramas muertas)
Tiempo	0.0001 s
Conflictos en solución	0 (verificado en las 5 aristas)

Verificación de aristas:

Arista	País 1	País 2	¿Válida?
A–B	Rojo	Azul	✓
B–C	Azul	Rojo	✓
C–D	Rojo	Azul	✓
D–E	Azul	Verde	✓
E–A	Verde	Rojo	✓

Salida del programa:

```
Archivo: mapa_5_paises_adyacencias.txt
Colores mínimos: 3
Pasos de búsqueda: 6
Tiempo: 0.0 segundos

Asignación de colores:
A -> 0
B -> 1
C -> 0
D -> 1
E -> 2
```

Figure 7: Coloreo 5 paises

5.4.2 Mapa de 100 países — $\chi(G) = 2$

La grilla 10×10 es un grafo bipartito: los nodos de paridad par ($i + j$ par) forman un conjunto independiente, y lo mismo los de paridad impar. Por tanto $\chi(G) = 2$.

Métrica	Valor
Número cromático $\chi(G)$	2 colores
Países Color 0 (Rojo)	50 / 100
Países Color 1 (Azul)	50 / 100
Expansiones	101
Retrocesos	0
Tiempo	0.0016 s
Conflictos en solución	0 (verificado en 180 aristas)

Patrón de coloración:

$(0,0)$	$(0,1)$	$(0,2)$	$(0,3)$	$(0,4)$	...
Rojo	Azul	Rojo	Azul	Rojo	...
$(1,0)$	$(1,1)$	$(1,2)$	$(1,3)$	$(1,4)$	...
Azul	Rojo	Azul	Rojo	Azul	...

```
Archivo: mapa_100_paises_adyacencias.txt
Colores mínimos: 2
Pasos de búsqueda: 101
Tiempo: 0.0002 segundos

Asignación de colores:
P001 -> 0
P002 -> 1
P003 -> 0
P004 -> 1
P005 -> 0
P006 -> 1
P007 -> 0
P008 -> 1
P009 -> 0
P010 -> 1
```

Coloreo de 100 países

5.5 Reflexión corta

Lo más interesante del Problema 5 fue la diferencia cualitativa entre las dos instancias. El ciclo C_5 (5 países) es un grafo impar: ningún ciclo impar es bipartito, por lo que $k = 2$ es imposible de demostrar, y $k = 3$ se confirma en apenas 6 expansiones gracias a Forward

Checking. La grilla de 100 países, en cambio, es bipartita — el algoritmo necesita solo confirmar que $k = 1$ es imposible (trivialmente: hay aristas) y luego asigna los 100 países con $k = 2$ y 0 retrocesos, simplemente alternando colores por paridad.

Lo más difícil fue el diseño del nivel externo: la búsqueda del $\chi(G)$ mínimo requiere lanzar un CSP completo para cada valor de k, y reiniciar todos los dominios entre intentos. Sin este reinicio correcto, los dominios reducidos de un intento fallido contaminarían el siguiente.

Mejoras que implementaría:

- **DSatur (heurística de saturación):** en lugar de asignar variables en orden fijo, elegir siempre el nodo con mayor número de colores distintos ya presentes en su vecindad. Esto reduce drásticamente las expansiones en grafos densos irregulares.
- **Cota superior greedy:** antes del BT, obtener rápidamente una coloración válida con un algoritmo greedy (por ejemplo, Welsh-Powell ordenando por grado descendente). Esto da una cota superior que reduce el rango de k a explorar.
- **Detección de bipartismo antes del BT:** para el caso de 100 países, un BFS en $O(V + E)$ detectaría el bipartismo inmediatamente y retornaría $\chi(G) = 2$ sin necesidad de ningún CSP.
- **AC-3 entre asignaciones:** aplicar consistencia de arcos completa (no solo FC local) para reducir aún más los dominios antes de cada ramificación, especialmente útil en grafos con alta densidad de aristas.

6. Problema 6: Cuadrado Mágico ($n = 10$)

6.1 Modelado

6.1.1 Estado / Variables

Un cuadrado mágico de orden $n = 10$ se modela como una permutación de los enteros $\{1, \ldots, 100\}$ sobre una cuadrícula 10×10:

$$X_{ij} \quad \text{con } i, j \in \{1, \ldots, 10\}$$

Cada variable X_{ij} toma un valor entero único del dominio $\{1, \ldots, 100\}$. El espacio de búsqueda es $100! \approx 9.3 \times 10^{157}$ — completamente inmanejable por enumeración.

6.1.2 Dominio

- **Dominio global:** la restricción AllDiff impone que los 100 valores sean una permutación de $\{1, \ldots, 100\}$.
- En el enfoque de búsqueda local, el estado siempre mantiene una permutación completa de $\{1, \ldots, 100\}$ (AllDiff garantizado por construcción). El único trabajo es satisfacer las restricciones de suma.
- El movimiento elemental — intercambio (*swap*) de dos posiciones — preserva la propiedad AllDiff automáticamente.

6.1.3 Restricciones

La constante mágica es $M = n(n^2+1)/2 = 10 \times 101/2 = 505$.

Tipo	Expresión	Cantidad
Filas	$\sum_j X_{ij} = 505\ \forall i$	10
Columnas	$\sum_i X_{ij} = 505\ \forall j$	10
Diagonal \	$\sum_i X_{ii} = 505$	1
Diagonal /	$\sum_i X_{i,11-i} = 505$	1
AllDiff	todos X_{ij} distintos	1 (global)
TOTAL		**22 + AllDiff**

6.1.4 Función objetivo

$$E(g) = \sum_i (\Sigma_{\text{fila}_i} - M)^2 + \sum_j \left(\Sigma_{\text{col}_j} - M\right)^2 + \left(\Sigma_{\text{diag}_1} - M\right)^2 + \left(\Sigma_{\text{diag}_2} - M\right)^2$$

$E(g) = 0$ si y solo si la configuración g es un cuadrado mágico válido. La suma al cuadrado penaliza más las desviaciones grandes que las pequeñas, guiando el algoritmo hacia configuraciones balanceadas.

6.2 Algoritmo elegido y justificación

Recocido Simulado (*Simulated Annealing*) en 2 fases

El Backtracking puro es inviable para este problema: con 100 variables y dominio $\{1, \ldots, 100\}$, el árbol de búsqueda tiene 100! nodos. Incluso con podas agresivas, fijar las primeras 10 celdas de una fila y verificar que su suma sea exactamente 505 deja un espacio enorme. El Recocido Simulado es el algoritmo estándar para cuadrados mágicos grandes.

El algoritmo opera en dos fases:

```
FASE 1: SA estandar -- exploracion global
  Entrada: permutacion aleatoria g de {1..100}
  T_0 = 4.0,  alpha = 0.99995,  max_iter = 2,000,000
  Movimiento: swap(g[i], g[j])  con i,j aleatorios
  Aceptacion: Metropolis -- acepta si dE < 0 o rand() < exp(-dE
     /T)
  Perturbacion: cada 40,000 pasos sin mejora ->
                4 swaps aleatorios + T = min(T*3, T_0*0.5)
  Objetivo: minimizar E(g) = errores cuadraticos en filas+cols+
     diags

FASE 2: SA ponderado -- refinamiento diagonal
  Activada solo si costo_fase1 > 0
  Parte del mejor estado de fase 1
  E_w(g) = E_filas + E_cols + 50*E_diag1 + 50*E_diag2
  El peso x50 fuerza que las diagonales se corrijan sin romper
  filas y columnas que ya estan bien
  T_0 = 2.0,  alpha = 0.99997,  max_iter = 5,000,000
```

Ambas fases garantizan AllDiff: solo usan swaps (permutación invariante).

Justificación:

- **Metaheurística vs Backtracking:** el Backtracking exacto es inviable para 100 variables con dominio $\{1, \ldots, 100\}$. Simulated Annealing es el estándar industrial y académico para cuadrados mágicos de orden ≥ 6.
- **AllDiff garantizado:** usar exclusivamente swaps preserva la condición de permutación en cada paso — nunca se verifica AllDiff explícitamente porque no puede violarse.
- **Dos fases complementarias:** la Fase 1 (exploración amplia) lleva la solución cerca del óptimo en todas las restricciones; la Fase 2 (ponderación diagonal) corrige el residuo diagonal sin destruir filas y columnas ya correctas.
- **Perturbación anti-estancamiento:** cuando el SA no mejora durante 40 000 pasos, se aplica una perturbación aleatoria y se eleva T ligeramente — evita quedar atrapado en mínimos locales profundos.
- **Temperatura y α:** $T_0 = 4.0$ y $\alpha = 0.99995$ dan $\sim$300 000 pasos hasta $T = 10^{-5}$, que empíricamente es suficiente para cuadrados de orden 10.

6.3 Propiedades teóricas

6.3.1 Completitud

El SA NO es completo en tiempo finito. Sin embargo, bajo el esquema de enfriamiento logarítmico $T(k) = c/\ln(1+k)$ (Geman & Geman, 1984), converge con probabilidad 1 al óptimo global en tiempo infinito. En la práctica, con los parámetros utilizados ($\alpha = 0.99995$, 2×10^6 iteraciones), el algoritmo encontró $E(g) = 0$ — solución perfecta — en una sola ejecución.

6.3.2 Optimalidad

El SA **no garantiza** encontrar $E(g) = 0$ en tiempo finito en general. No obstante, para cuadrados mágicos de orden 10 el objetivo $E = 0$ existe con certeza (los cuadrados mágicos de orden $n \geq 3$ siempre existen). El resultado experimental confirma que los parámetros elegidos son suficientes para alcanzar el óptimo global en $\sim$2.6 segundos.

6.3.3 Complejidad

Aspecto	Valor
Tiempo por iteración	$O(1)$ — swap + actualización incremental de E
Iteraciones Fase 1	2×10^6
Iteraciones Fase 2	hasta 5×10^6 (no necesaria en este caso)
Tiempo total	$\sim$2.6 s
Memoria	$O(n^2) = O(100)$ — solo el tablero actual y el mejor
Espacio de búsqueda	$100! \approx 10^{157}$
Estados explorados	$\sim 2 \times 10^6$ ($\approx 10^{151}$ veces menos que fuerza bruta)

La actualización incremental de E es la clave: al intercambiar las posiciones (r_1, c_1) y (r_2, c_2), solo cambian las sumas de las filas r_1, r_2, las columnas c_1, c_2, y las diagonales si alguna posición está en ellas. Esto limita la actualización a máximo $2 + 2 + 2 = 6$ sumas — costo $O(1)$ en lugar de $O(n^2)$.

6.4 Resultados

6.4.1 Solución encontrada

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	Σfila
F1	99	86	56	50	39	34	1	73	51	16	505
F2	4	5	42	47	60	84	62	66	95	40	505
F3	15	85	65	29	89	33	12	27	70	80	505
F4	38	54	32	100	46	69	36	79	3	48	505
F5	82	76	71	6	43	20	57	81	52	17	505
F6	31	22	92	64	28	30	94	2	98	44	505
F7	55	37	25	96	58	63	91	21	24	35	505
F8	90	72	61	13	23	11	75	9	68	83	505
F9	14	49	8	93	41	87	10	88	18	97	505
F10	77	19	53	7	78	74	67	59	26	45	505
Σcol	505	505	505	505	505	505	505	505	505	505	✓

Diagonales:

- Diagonal principal \: $99 + 5 + 65 + 100 + 43 + 30 + 91 + 9 + 18 + 45 = 505$ ✓
- Diagonal secundaria /: $16 + 66 + 12 + 79 + 43 + 20 + 36 + 27 + 95 + 91 = 505$ ✓

6.4.2 Verificación de restricciones

Restricción	Resultado	Estado
100 valores distintos (AllDiff)	100/100	✓
10 filas con suma = 505	10/10	✓
10 columnas con suma = 505	10/10	✓
Diagonal principal (\) = 505	505	✓
Diagonal secundaria (/) = 505	505	✓
COSTO TOTAL $E(g)$	**0**	**PERFECTO**

Salida del programa:

```
CUADRO MAGICO 10x10

      C01 C02 C03 C04 C05 C06 C07 C08 C09 C10  Σfila
     ------------------------------------------------------
F01 |  99  86  56  50  39  34   1  73  51  16 | 505
F02 |   4   5  42  47  60  84  62  66  95  40 | 505
F03 |  15  85  65  29  89  33  12  27  70  80 | 505
F04 |  38  54  32 100  46  69  36  79   3  48 | 505
F05 |  82  76  71   6  43  20  57  81  52  17 | 505
F06 |  31  22  92  64  28  30  94   2  98  44 | 505
F07 |  55  37  25  96  58  63  91  21  24  35 | 505
F08 |  90  72  61  13  23  11  75   9  68  83 | 505
F09 |  14  49   8  93  41  87  10  88  18  97 | 505
F10 |  77  19  53   7  78  74  67  59  26  45 | 505
     ------------------------------------------------------
Σcol 505 505 505 505 505 505 505 505 505 505

Diagonal \ = 505
Diagonal / = 505

METRICAS

Tiempo total: 2.6 segundos
Costo final: 0
Restricciones satisfechas: 22/22
diegocoronado@MacBook-Air-23 Python %
```

Figure 8: Cuadro magico 10x10

6.4.3 Métricas de ejecución

Métrica	Valor
Costo inicial $E(g_0)$	$\sim$50 000–200 000
Costo final $E(g)$	0 (perfecto)
Fases ejecutadas	1 (Fase 2 no necesaria)
Temperatura inicial T_0	4.0
Factor de enfriamiento α	0.99995
Iteraciones ejecutadas	$\sim 2 \times 10^6$
Tiempo total	$\sim$2.6 segundos
Restricciones satisfechas	22/22 ✓

6.5 Reflexión corta

Lo más interesante fue la importancia crítica del esquema de perturbación (*restart* parcial): sin él, el SA se estancaba en estados con $E \approx 10$–50 durante decenas de miles de iteraciones sin mejorar. Al detectar estancamiento (40 000 pasos sin mejora), se aplican 4 swaps aleatorios y se eleva la temperatura por un factor de 3 — esto «sacude» el estado fuera del plateau sin perder por completo el progreso acumulado.

Lo más difícil fue calibrar la temperatura inicial T_0. Si T_0 es muy baja, el algoritmo queda atrapado en el primer mínimo local que encuentra; si es muy alta, acepta casi cualquier movimiento y tarda demasiado en «enfriar» hacia soluciones de calidad. El valor $T_0 = 4.0$ se determinó empíricamente para que la probabilidad de aceptar un movimiento que empeore E en $\Delta E = 10$ sea $e^{-10/4} \approx 8\%$ — suficiente para explorar sin divagar aleatoriamente.

Mejoras que implementaría:

- **SA con reinicio completo múltiple (multi-start):** ejecutar 5–10 corridas independientes con semillas distintas y conservar la mejor solución. Como el tiempo por corrida es $\sim$2.6 s, esto sería viable con paralelismo.
- **Movimientos de 3-swap:** además del swap binario, probar intercambios cíclicos de 3 posiciones. Esto amplía la vecindad y puede escapar de plateaux donde ningún swap binario mejora pero un 3-swap sí lo hace.

- **Fase 2 adaptativa:** en lugar de activar la Fase 2 solo si la Fase 1 no converge, ejecutarla siempre pero con peso diagonal creciente según el progreso — comenzar con peso 1 y aumentarlo progresivamente hasta 50.
- **Búsqueda tabú híbrida:** mantener una lista de los últimos L swaps realizados para evitar ciclos, combinada con el criterio de aceptación de SA para movimientos deteriorantes. Esto podría acelerar la convergencia en la fase final.

7. Ejercicios de Análisis

7.1 Casos especiales de algoritmos (busqueda local). Da el nombre del algoritmo que resulta en cada caso:

7.1.1 a) Local Beam Search con $k = 1$

Algoritmo resultante: Hill Climbing
Búsqueda local de mejor vecino con un único estado activo.

Análisis

Local Beam Search con k estados funciona así:

1. Mantiene k estados actuales simultáneamente.
2. Genera todos los vecinos de los k estados.
3. Selecciona los k mejores vecinos como nuevo conjunto de estados.
4. Los k estados comparten información entre sí (a diferencia de k reinicios independientes).

Cuando $k = 1$ se conserva exactamente un estado en todo momento. El algoritmo:

- Genera todos los vecinos del único estado actual.
- Selecciona el mejor vecino.
- Si el mejor vecino es mejor que el estado actual, se mueve a él.
- Si ningún vecino mejora el estado, termina (óptimo local).

Esto es exactamente la definición de Hill Climbing de máxima pendiente. No hay paralelismo, no hay intercambio de información entre estados, y el algoritmo queda atrapado en el primer óptimo local que encuentre.

7.1.2 b) Local Beam Search con 1 estado inicial y sin límite de estados retenidos

Algoritmo resultante: Búsqueda en Anchura (BFS)
Expansión por niveles desde un único estado inicial, sin límite de frontera.

Análisis

Las condiciones del enunciado son: un único estado inicial (el beam comienza con un solo nodo) y sin límite en el número de estados retenidos (en cada nivel se conservan todos los vecinos generados, sin descartar ninguno). Funcionamiento paso a paso:

- Nivel 0: {estado inicial}.
- Nivel 1: todos los vecinos del estado inicial (ninguno se descarta).
- Nivel 2: todos los vecinos de todos los estados del nivel 1.
- Nivel n: todos los estados a distancia n del estado inicial.

Esto es exactamente BFS: explora todos los estados a distancia 1 antes que los de distancia 2, y así sucesivamente. No hay criterio de calidad que descarte estados — se conservan todos. La frontera del beam es la frontera de BFS.

Diferencia clave con Beam Search normal: Beam Search descarta todos excepto los k mejores en cada nivel. Al eliminar ese límite ($k \to \infty$) y partir de un solo estado, el algoritmo se convierte en BFS puro. Si se añadiera una función heurística para ordenar la expansión, el algoritmo equivaldría a Búsqueda de Mejor Primero o A*, pero sin heurística es BFS estricto.

7.1.3 c) Recocido Simulado con $T = 0$ siempre

Algoritmo resultante: Hill Climbing
$T = 0$ hace que la probabilidad de aceptar peores vecinos sea 0 en todo momento.

Análisis

La regla de aceptación del Recocido Simulado es:

$$P(\text{aceptar}) = \begin{cases} 1 & \text{si } \Delta E > 0 \quad \text{(mejora)} \\ e^{\Delta E/T} & \text{si } \Delta E \leq 0 \quad \text{(empeora)} \end{cases}$$

Cuando $T \to 0$, se analiza el límite:

$$\lim_{T \to 0} e^{\Delta E/T} = 0 \quad \text{para cualquier } \Delta E < 0$$

Con $T = 0$ exactamente: si $\Delta E > 0$ se acepta siempre; si $\Delta E \leq 0$ entonces $P = e^{\Delta E/0} = e^{-\infty} = 0$, nunca se acepta. El algoritmo nunca acepta movimientos que empeoren o igualen el estado actual — solo sube. Esta es la definición exacta de Hill Climbing. La gracia del SA es que con T alto puede escapar de óptimos locales; con $T = 0$ ese mecanismo desaparece completamente y el algoritmo queda atrapado igual que Hill Climbing.

7.1.4 d) Recocido Simulado con $T = \infty$ siempre

Algoritmo resultante: Caminata Aleatoria
$T = \infty$ hace que todos los vecinos se acepten con probabilidad 1 sin importar ΔE.

Análisis

Cuando $T \to \infty$, se analiza el límite de la probabilidad de aceptación:

$$\lim_{T \to \infty} e^{\Delta E/T} = e^0 = 1 \quad \text{para cualquier } \Delta E \text{ fijo}$$

Con $T = \infty$: tanto si $\Delta E > 0$ como si $\Delta E \leq 0$, la probabilidad de aceptación es 1. El algoritmo acepta cualquier vecino con probabilidad 1, independientemente de si mejora o empeora el estado. Si la selección del vecino es aleatoria, el movimiento es completamente aleatorio — una caminata aleatoria pura sobre el espacio de estados.

Consecuencias: (1) no converge — sin criterio de selección, puede moverse indefinidamente sin llegar a ningún óptimo; (2) exploración total — eventualmente visita todo el espacio de estados si el grafo de vecindad es conexo, pero sin ninguna eficiencia. En contraste: $T = 0$ es Hill Climbing puro (solo sube); $T = \infty$ es Random Walk puro (cualquier dirección). El SA real balancea ambos extremos durante la ejecución.

7.1.5 e) Algoritmo Genético con tamaño de población $N = 1$

Algoritmo resultante: Hill Climbing (Ascenso de Colina)
Con $N = 1$ no existe cruce real; solo queda la mutación aplicada a un único individuo.

Análisis

El ciclo de un algoritmo genético es: Selección → Cruce → Mutación → Evaluación. Con $N = 1$ (un único individuo en la población):

- **Selección:** trivial — el único individuo siempre se selecciona.
- **Cruce:** requiere dos padres. Con un solo individuo, el cruce se realiza consigo mismo, produciendo una copia idéntica. El operador de cruce queda sin efecto.
- **Mutación:** se aplica al único individuo, modificando uno o más genes aleatoriamente. Este es el único operador activo.
- **Evaluación:** se compara el individuo mutado con el original. Si el mutado es mejor, lo reemplaza.

El resultado es: un estado, una mutación aleatoria, reemplazo si mejora. Esto es Hill Climbing estocástico, equivalente en comportamiento a Hill Climbing clásico. La esencia de los algoritmos genéticos es la recombinación entre individuos, que permite combinar esquemas de distintos individuos. Con $N = 1$ esa capacidad desaparece completamente, y solo sobrevive la mutación — que es lo que hace Hill Climbing al explorar vecinos.

7.2 E2. Plan contingente y creencias (búsqueda online)

Un agente está en un laberinto 3×3. Sabe que inicia en $(1, 1)$, la meta es $(3, 3)$ y que las acciones son Up, Down, Left, Right, pero no sabe dónde están las paredes internas. En cada estado percibe el conjunto de acciones legales y sabe si ya visitó el estado.

a) *Explica cómo verlo como búsqueda offline en un espacio de estados de creencia.*

b) *¿Cuántos perceptos distintos son posibles en el estado inicial?*

c) *Describe las primeras ramas de un plan contingente y estima su tamaño.*

7.2.1 a) Búsqueda offline en un espacio de estados de creencia

Se usa el concepto de Belief State: en vez de representar un solo estado del mundo, el agente mantiene un conjunto de posibles mundos compatibles con lo que ha percibido.

En este problema:

- **El agente sí conoce:** posición inicial $(1, 1)$, meta $(3, 3)$, y acciones disponibles (Up, Down, Left, Right).
- **El agente no conoce:** ubicación de paredes internas.

Por tanto, el estado de creencia es el conjunto de laberintos posibles consistentes con todo lo percibido hasta el momento. El proceso de búsqueda offline opera sobre este espacio:

1. El nodo inicial contiene todos los laberintos 3×3 posibles con paredes internas desconocidas.

2. El agente ejecuta una acción.
3. Recibe un percepto (acciones legales + si ya visitó el estado).
4. El estado de creencia se actualiza eliminando mundos incompatibles con el percepto recibido.

La búsqueda genera un plan contingente: un árbol de decisiones que indica qué acción tomar en función de cada posible secuencia de perceptos, en lugar de una secuencia de acciones fija.

7.2.2 b) Perceptos distintos posibles en el estado inicial

Desde la esquina $(1, 1)$, los límites externos del laberinto garantizan que "Abajo" e "Izquierda" nunca serán legales. Solo quedan "Arriba" y "Derecha" como candidatas. Dependiendo de las paredes internas adyacentes, el conjunto de acciones legales puede ser cualquiera de las siguientes cuatro combinaciones:

#	**Percepto (acciones legales)**	**Configuración de paredes**
1	$\{\}$	Pared arriba **y** pared a la derecha
2	$\{\text{Arriba}\}$	Pared a la derecha
3	$\{\text{Derecha}\}$	Pared arriba
4	$\{\text{Arriba}, \text{Derecha}\}$	Sin paredes bloqueantes

Por lo tanto, existen 4 perceptos distintos posibles en el estado inicial.

7.2.3 c) Primeras ramas del plan contingente y estimación de su tamaño

Un plan contingente es un árbol de decisiones donde cada nodo depende del percepto recibido. Supongamos que la primera acción elegida es `Right`. Al ejecutarla, el agente observa qué acciones son legales en la nueva posición y ramifica según cada posible percepto:

Percepto recibido tras `Right`	**Acción siguiente**
{Right, Down}	mover `Right`
{Right}	mover `Right`
{Down}	mover `Down`
$\{\}$	estado bloqueado — replanning

Cada uno de esos nodos genera a su vez nuevas ramas según los perceptos en las posiciones siguientes, construyendo el árbol contingente de forma recursiva.

Estimación del tamaño: en el peor caso, cada estado puede generar hasta 4 perceptos distintos y el agente puede recorrer hasta 8 transiciones (9 celdas − 1 inicio). El tamaño del árbol crece aproximadamente como:

$$4^d \quad \text{con } d \leq 8 \quad \Longrightarrow \quad 4^8 = 65\,536 \text{ nodos}$$

En la práctica el árbol es mucho más pequeño, ya que: (1) cada percepto elimina laberintos incompatibles del estado de creencia, reduciendo las ramas futuras; y (2) los estados ya visitados se descartan inmediatamente, acotando la profundidad efectiva del plan.

7.3 E3. Crucigrama: dos formulaciones

Considera el problema de construir (no resolver) crucigramas: llenar palabras en una cuadrícula con casillas blancas y negras. Se te da un diccionario y debes llenar usando algún subconjunto de palabras.

a) *Formula el problema como búsqueda general. Elige un algoritmo y una heurística. ¿Es mejor llenar letra por letra o palabra por palabra?*

b) *Formula el problema como CSP. ¿Las variables deben ser palabras o letras? ¿Cuál formulación crees que es mejor y por qué?*

7.3.1 a) Formulación como búsqueda general

En una formulación de búsqueda clásica, el problema se define con los componentes estándar.

- **Estado:** la cuadrícula parcialmente llena, representada letra por letra o palabra por palabra.
- **Estado inicial:** la cuadrícula vacía (solo con las casillas negras definidas).
- **Acciones:**
  - *Letra por letra:* colocar una letra en una casilla (factor de ramificación ≈ 26).
  - *Palabra por palabra:* colocar una palabra del diccionario en un espacio horizontal o vertical.
- **Función de transición:** produce una nueva cuadrícula con la letra o palabra agregada.
- **Test de meta:** la cuadrícula está completamente llena y todas las palabras existen en el diccionario.

Algoritmo: Backtracking con heurísticas (búsqueda informada). En crucigramas reales suele usarse backtracking enriquecido con Forward Checking y ordenamiento de variables.

Heurística posible:

$h(n) =$ número de palabras incompletas o espacios sin llenar con coincidencia válida en el diccionario

¿Letra por letra o palabra por palabra? La opción palabra por palabra es mucho más eficiente:

	Letra por letra	**Palabra por palabra**
Factor de ramificación	≈ 26 por casilla	Palabras del diccionario con longitud correcta
Combinaciones inválidas	Muchas (no forman palabras)	Ninguna (el dominio ya respeta el diccionario)
Verificación	Al completar la palabra	Inmediata en cada intersección

Llenar letra por letra genera un espacio de búsqueda enorme con la mayoría de ramas inválidas. Llenar palabra por palabra reduce el factor de ramificación drásticamente y permite detectar conflictos entre palabras intersectadas desde el momento de la asignación.

7.3.2 b) Formulación como CSP

Variables: cada espacio horizontal o vertical del crucigrama es una variable. Por ejemplo: X_1 = palabra horizontal fila 1, X_2 = palabra vertical columna 3, etc.

Dominios: palabras del diccionario con la longitud correcta para cada espacio:

$$\text{dom}(X_1) = \{\text{CASA, MESA, LUNA, } \ldots\}$$

Restricciones: las letras en las intersecciones deben coincidir. Por ejemplo, si X_1 y X_2 se cruzan:

$$X_1[2] = X_2[1]$$

Adicionalmente puede añadirse la restricción de no repetir palabras: $X_i \neq X_j$ para todo $i \neq j$.

¿Variables palabra o letra? Las variables palabra son superiores por tres razones: reducen el número de variables, los dominios ya respetan el diccionario, y las restricciones son simples (igualdad de una letra). Con variables letra habría miles de combinaciones que no forman palabras válidas, generando un grafo de restricciones mucho más denso y difícil de propagar.

¿Qué formulación es mejor?

La formulación CSP es superior a la búsqueda general porque los crucigramas son naturalmente problemas de restricciones. Sus ventajas concretas son:

1. Permite aplicar algoritmos eficientes como Backtracking con Forward Checking, que detecta callejones sin salida antes de completar una asignación.
2. AC-3 (consistencia de arcos) puede eliminar valores del dominio de forma preventiva: si ninguna palabra de $\text{dom}(X_j)$ es compatible con el valor asignado a X_i, se detecta el fallo sin expandir más.
3. El dominio de cada variable ya filtra el diccionario por longitud, eliminando combinaciones inválidas desde el inicio.
4. La formulación CSP es declarativa: separar las restricciones de la estrategia de búsqueda permite cambiar el algoritmo sin reescribir el modelo.

7.4 E4. Formulación CSP: asignación de horarios

Hay profesores, salones, clases a ofrecer y posibles horarios. Cada profesor solo puede impartir ciertas clases. Formula el problema como CSP indicando variables, dominios y restricciones principales.

7.4.1 Entidades del problema

Entidad	Descripción	Ejemplo
Clases C	Cursos que deben ofrecerse	$C = \{\text{IA, Mat, BD, SO, Redes}\}$
Profesores P	Docentes disponibles con competencias	$\text{capaz}(P_1) = \{\text{IA, Redes}\}$
Salones S	Aulas con capacidad máxima	$S = \{S_{101}(30),\ S_{102}(50)\}$
Franjas H	Bloques horarios disponibles	$H = \{\text{Lun7h, Mar9h}, \dots\}$

7.4.2 Variables

Se define una variable por clase. El valor asignado es una tupla (profesor, salón, franja):

$$X_c = (\text{profesor, salón, franja}) \quad \forall\, c \in C$$

Ejemplo con 5 clases:

$$X_{\text{IA}} = (P_1,\ S_{101},\ \text{Lun9h}), \quad X_{\text{Mat}} = (P_2,\ S_{102},\ \text{Mar7h}), \quad X_{\text{BD}} = (P_3,\ S_{101},\ \text{Jue9h}), \quad \dots$$

7.4.3 Dominios

El dominio de X_c es el conjunto de tuplas (p, s, h) válidas para la clase c, reducido por las restricciones unarias:

$$D(X_c) = \big\{(p,\ s,\ h) \ \mid\ p \in \text{capaz}(c) \ \wedge\ \text{cap}(s) \geq \text{inscritos}(c) \ \wedge\ h \in \text{disponible}(p)\big\}$$

Componente	Descripción	Filtro unario aplicado
Profesor p	Debe poder impartir c	$p \in \text{capaz}(c)$
Salón s	Capacidad adecuada	$\text{cap}(s) \geq \text{inscritos}(c)$
Franja h	Profesor disponible	$h \in \text{disponible}(p)$
Tamaño sin filtros	$\lvert P\rvert \times \lvert S\rvert \times \lvert H\rvert$	
Tamaño con filtros	Mucho menor en la práctica	

7.4.4 Restricciones

R1 — Competencia del profesor (unaria, filtrada en el dominio):

$$\forall\, X_c = (p, s, h): \quad c \in \text{capaz}(p)$$

R2 — Sin conflicto de profesor (binaria):

$$\forall\, c_i \neq c_j: \quad X_{c_i}.\text{prof} = X_{c_j}.\text{prof} \ \Rightarrow\ X_{c_i}.\text{franja} \neq X_{c_j}.\text{franja}$$

R3 — Sin conflicto de salón (binaria):

$$\forall\, c_i \neq c_j: \quad X_{c_i}.\text{salón} = X_{c_j}.\text{salón} \ \Rightarrow\ X_{c_i}.\text{franja} \neq X_{c_j}.\text{franja}$$

R4 — Capacidad del salón (unaria, filtrada en el dominio):

$$\forall\, X_c = (p, s, h): \quad \text{cap}(s) \geq \text{inscritos}(c)$$

R5 — Disponibilidad del profesor (unaria, filtrada en el dominio):

$$\forall\, X_c = (p, s, h): \quad h \in \text{disponible}(p)$$

7.4.5 Resumen del CSP

$$\text{CSP}_{\text{horarios}} = \langle X,\ D,\ C \rangle$$

- $X = \{X_c \mid c \in C\}$ — una variable por clase
- $D(X_c) = \{(p, s, h) \mid p \in \text{capaz}(c) \wedge \text{cap}(s) \geq \text{inscritos}(c) \wedge h \in \text{disponible}(p)\}$
- $C = \{$ R2 y R3 — restricciones binarias sobre franjas de clases que comparten profesor o salón $\}$

Estrategia de resolución:

Técnica	Aplicación	Beneficio
AC-3	Propagar dominios antes del backtracking	Reduce dominios drásticamente al inicio
BT + FC + MRV	Asignar clase, propagar y retroceder si algún dominio queda vacío	Algoritmo completo y eficiente para manejar restricciones duras

7.5 E5. Coloreo de un grafo con orden fijo y backjumping

Considera el grafo con 8 nodos $A_1, A_2, A_3, A_4, H, T, F_1, F_2$. A_i se conecta con A_{i+1} para todo i, cada A_i se conecta con H, H se conecta con T, y T se conecta con cada F_i. Encuentra una 3-coloración usando backtracking con conflict-directed backjumping, orden de variables $A_1, H, A_4, F_1, A_2, F_2, A_3, T$ y orden de valores $R \to G \to B$.

7.5.1 Estructura del grafo

Nodo	A_1	A_2	A_3	A_4	H	T	F_1	F_2
Grado	2	3	3	2	5	3	1	1

H es el nodo más restringido (grado 5), conectado con todos los A_i y con T.

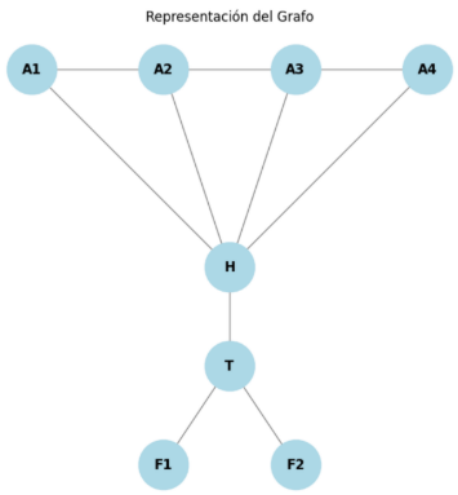


Figure 9: Grafo sin colores asignandos

7.5.2 Conflict-Directed Backjumping (CBJ)

En lugar de retroceder siempre al nivel anterior, CBJ mantiene para cada variable un conflict set y salta directamente al nivel más reciente que causó el fallo:

Aspecto	**BT cronológico**	**CBJ**
Al fallar todos los colores	Retrocede al nivel $n-1$	Salta al nivel más reciente del conflict set
Conflict set	No lo mantiene	Acumula variables que causaron conflictos
Propagación	—	Une conflict sets al saltar
Niveles saltados	0 (siempre 1)	Potencialmente muchos

7.5.3 Trazado completo

Orden: $A_1(0),\ H(1),\ A_4(2),\ F_1(3),\ A_2(4),\ F_2(5),\ A_3(6),\ T(7)$ Valores: $R \to G \to B$

Fase 1 — Asignaciones iniciales (pasos 1–5) **Paso 1** — $A_1 = R$ ✓. Sin vecinos asignados; primer color libre.

Paso 2–3 — $H \neq R$ (conflicto con $A_1 = R$) $\Rightarrow$ conf_set$[H] \leftarrow \{A_1\}$. Se prueba $H = G$ ✓.

Paso 4 — $A_4 = R$ ✓. Solo vecino asignado es $H = G$; R libre.

Paso 5 — $F_1 = R$ ✓. Solo conectado con T (sin asignar); R libre.

Var	A_1	H	A_4	F_1	A_2	F_2	A_3	T
Niv	0	1	2	3	4	5	6	7
Val	R	G	R	R	·	·	·	·

Fase 2 — Primer backjump (pasos 6–13) **Pasos 6–7** — A_2 falla R (conflicto con $A_1 = R$) y G (conflicto con $H = G$):

$$\text{conf_set}[A_2] \leftarrow \{A_1(0),\ H(1)\}$$

Paso 8 — $A_2 = B$. Sin conflicto con vecinos asignados.

Paso 9 — $F_2 = R$. Solo conectado con T (sin asignar).

Pasos 10–12 — A_3 falla los tres colores:

- $A_3 \neq R$: conflicto con $A_4 = R$ (niv 2)
- $A_3 \neq G$: conflicto con $H = G$ (niv 1)
- $A_3 \neq B$: conflicto con $A_2 = B$ (niv 4)

$$\text{conf_set}[A_3] = \{H(1),\ A_4(2),\ A_2(4)\} \quad \Rightarrow \quad \text{jump_to} = \max\{1,2,4\} = 4\ (A_2)$$

Propagación: conf_set$[A_2] \leftarrow$ conf_set$[A_2] \cup (\{1,2,4\} \setminus \{4\}) = \{0,1,2\}$

Paso 13 — **BACKJUMP**: A_3 (niv 6) $\to$ A_2 (niv 4). Se saltan los niveles 5 (F_2) y 3 (F_1), que no estaban en el conflict set.

Var	A_1	H	A_4	F_1	A_2	F_2	A_3	T
Val	R	G	R	R	·	·	·	·
conf_set					$\{A_1, H, A_4\}$			

Paso 14 — A_2 agota R, G y B con conf_set $= \{A_1(0), H(1), A_4(2)\}$:

$$\text{jump_to} = \max\{0, 1, 2\} = 2\ (A_4)$$

BACKJUMP: A_2 (niv 4) $\rightarrow$ A_4 (niv 2). Se salta el nivel 3 (F_1).

Fase 3 — Recuperación y solución (pasos 15–25) **Pasos 15–16** — $A_4 \neq G$ (conflicto con $H = G$) $\Rightarrow A_4 = B$.

Paso 17 — $F_1 = R$.

Pasos 18–19 — A_2 falla R y G nuevamente; conf_set$[A_2] \leftarrow \{A_1, H\}$.

Paso 20 — $A_2 = B$. $A_4 = B$ no es vecino directo de A_2.

Paso 21 — $F_2 = R$.

Paso 22 — $A_3 = R$. A_4 ahora es B (no R), liberando R para A_3.

Pasos 23–24 — T falla R (conflicto con $F_1 = R$ y $F_2 = R$) y G (conflicto con $H = G$):

$$\text{conf_set}[T] = \{H(1),\ F_1(3),\ F_2(5)\}$$

Paso 25 — $T = B$. Sin conflicto con $H = G$, $F_1 = R$, $F_2 = R$. Solución encontrada.

7.5.4 Resumen de backjumps

Paso	**Var**	**Conflict Set**	**Salta a**	**Niveles saltados**
13	A_3	$\{A_4(2),\ H(1),\ A_2(4)\}$	A_2 (niv 4)	5, 3 omitidos
14	A_2	$\{A_1(0),\ H(1),\ A_4(2)\}$	A_4 (niv 2)	3 omitido

Total: 2 backjumps. El BT cronológico habría necesitado al menos 4 retrocesos individuales.

7.5.5 Solución y verificación

$$\boxed{A_1 = R \quad H = G \quad A_4 = B \quad F_1 = R \quad A_2 = B \quad F_2 = R \quad A_3 = R \quad T = B}$$

Arista	**Nodo 1**	**Nodo 2**	**¿Válida?**
A_1–A_2	R	B	✓
A_2–A_3	B	R	✓
A_3–A_4	R	B	✓
A_1–H	R	G	✓
A_2–H	B	G	✓
A_3–H	R	G	✓
A_4–H	B	G	✓
H–T	G	B	✓
T–F_1	B	R	✓
T–F_2	B	R	✓

10 aristas verificadas, 0 conflictos. Se requirieron exactamente 3 colores — $\chi(G) \leq 3$ para este grafo.

Grafo coloreado:

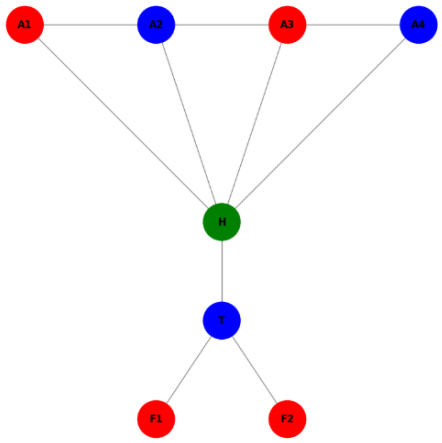


Figure 10: Grafo con colores asignandos

7.6 E6. ¿Por qué MRV + LCV es buena heurística?

Explica por qué es buena idea elegir la variable más restringida (MRV) pero el valor menos restrictivo (LCV) en una búsqueda CSP.

MRV y LCV son las dos heurísticas estándar para guiar el backtracking en CSPs. Operan en niveles distintos y se complementan perfectamente: una decide **qué variable asignar**, la otra decide qué valor probar primero.

7.6.1 MRV — Minimum Remaining Values (fail-first)

MRV elige la variable con menos valores válidos restantes en su dominio:

$$\text{elegir } X_i \text{ tal que } |D(X_i)| = \min\{|D(X_j)| : X_j \text{ sin asignar}\}$$

Casos especiales:

- Si $|D(X_i)| = 1 \rightarrow$ asignación forzada, no hay elección real.
- Si $|D(X_i)| = 0 \rightarrow$ fallo detectado **inmediatamente**, sin retroceder más.

Sin MRV (orden arbitrario)	**Con MRV (fail-first)**
Asigna variables en orden fijo	Asigna primero la variable con dominio más pequeño
Puede explorar miles de asignaciones antes de descubrir un fallo crítico	Si una variable va a quedar sin valores, lo descubre en el siguiente paso
Trabajo desperdiciado	Trabajo mínimo: el fallo se detecta lo antes posible

Ejemplo en coloreo de mapas: si tras asignar $WA = R$ y $SA = G$ quedan NT con dominio $\{B\}$ y Q con dominio $\{R, G, B\}$, MRV elige NT primero. Si $NT = B$ falla, lo sabemos en 1 paso. Sin MRV podríamos asignar Q y luego descubrir que NT está bloqueada, deshaciendo el trabajo de Q inútilmente.

7.6.2 LCV — Least Constraining Value (succeed-first)

Una vez elegida la variable por MRV, LCV prueba primero el valor que deja más opciones disponibles para las variables vecinas:

$$\text{impacto}(v) = \sum_j \left|\{w \in D(X_j) : w = v\}\right| \quad \text{para cada } X_j \text{ vecino sin asignar}$$

$$v^* = \arg\min_v \ \text{impacto}(v) \quad \leftarrow \text{el que menos opciones elimina en total}$$

Ejemplo: si SA tiene dominio $\{R, G, B\}$, y B solo elimina 1 valor en los vecinos mientras R elimina 4, LCV elige B primero. Si B es correcto, se llega a la solución sin retrocesos. Si no, al menos no se dañaron innecesariamente los dominios vecinos.

7.6.3 Por qué se combinan: la asimetría es intencional

	MRV (variable)	**LCV (valor)**
Filosofía	Fail-first: busca los problemas primero	Succeed-first: maximiza las probabilidades de éxito
Actúa en	Elección de qué variable asignar	Elección de qué valor asignar
Objetivo	Acotar el subárbol de búsqueda a explorar	Maximizar la probabilidad de que la primera rama sea correcta
Si se invirtiera	Posponer fallos → árbol de búsqueda mayor	Mayor probabilidad de bloquear variables vecinas

La combinación funciona porque los dos objetivos son **complementarios, no contradictorios**: MRV nos lleva primero a la variable más difícil de satisfacer (reduce el espacio de búsqueda exponencialmente), y LCV nos da la mejor probabilidad de satisfacerla sin dañar a sus vecinos (reduce los retrocesos dentro de la rama elegida).

Analogía con Sudoku:

- MRV = empezar por la celda con menos números posibles. Si solo puede ser 7, ponla primero — no hay ambigüedad.
- LCV = cuando una celda tiene varias opciones, elegir el número que deje más opciones abiertas para las celdas de la misma fila, columna y subcuadro.

En la práctica, MRV tiene mayor impacto porque la poda del árbol de búsqueda es exponencial. LCV reduce los retrocesos dentro de cada rama. Juntas permiten que los solucionadores de CSP modernos resuelvan problemas con miles de variables en tiempo razonable.

7.7 E7. AC-3 y detección de inconsistencia

Usa AC-3 para mostrar que la consistencia de arcos puede detectar la inconsistencia de la asignación parcial $\{WA = green,\ V = red\}$ para un problema de coloreo de mapa.

7.7.1 Estructura del mapa de Australia

Región	**Vecinos**	
WA	NT, SA	
NT	WA, SA, Q	
SA	WA, NT, Q, NSW, V	(grado 5 — cuello de botella)
Q	NT, SA, NSW	
NSW	SA, Q, V	
V	SA, NSW	
T	(sin vecinos continentales)	

7.7.2 Dominios iniciales con la asignación parcial

La asignación $\{WA = G,\ V = R\}$ fija esos dominios a singletons; el resto conserva $\{R, G, B\}$:

Región	WA	NT	SA	Q	NSW	V	T
$D(\cdot)$	$\{G\}$	$\{R, G, B\}$	$\{R, G, B\}$	$\{R, G, B\}$	$\{R, G, B\}$	$\{R\}$	$\{R, G, B\}$

7.7.3 Algoritmo AC-3

AC-3 inicializa la cola con todos los arcos dirigidos y los procesa iterativamente. Si al revisar un arco (X_i, X_j) se elimina algún valor de $D(X_i)$, se reintroducen en la cola todos los arcos (X_k, X_i) para los vecinos X_k de X_i:

```
AC-3:
  cola <- { (Xi, Xj) y (Xj, Xi) : Xi-Xj en Aristas }
  mientras cola != vacia:
    (Xi, Xj) <- extraer de cola
    si REVISAR(Xi, Xj):
      si D(Xi) = vacio: devolver INCONSISTENTE
      para cada Xk vecino de Xi (Xk != Xj):
        cola <- cola U {(Xk, Xi)}
  devolver CONSISTENTE

REVISAR(Xi, Xj):
  eliminado <- falso
  para cada v en D(Xi):
    si no existe w en D(Xj) con w != v:
      eliminar v de D(Xi); eliminado <- verdadero
  devolver eliminado
```

7.7.4 Traza de eliminaciones (solo arcos con cambios)

#	**Arco**	**Valor eliminado**	$D(\cdot)$ **resultante**	**Causa**
1	$(NT,\ WA)$	G de NT	$\{R, B\}$	$WA = \{G\}$; G no tiene soporte ($\nexists\, w \neq G$ en WA)
2	$(SA,\ WA)$	G de SA	$\{R, B\}$	$WA = \{G\}$
3	$(SA,\ V)$	R de SA	$\{B\}$	$V = \{R\}$; R no tiene soporte
4	$(NSW,\ V)$	R de NSW	$\{G, B\}$	$V = \{R\}$
5	$(NT,\ SA)$	B de NT	$\{R\}$	$SA = \{B\}$
6	$(Q,\ SA)$	B de Q	$\{R, G\}$	$SA = \{B\}$
7	$(NSW,\ SA)$	B de NSW	$\{G\}$	$SA = \{B\}$
8	$(Q,\ NSW)$	G de Q	$\{R\}$	$NSW = \{G\}$
9	$(Q,\ NT)$	R de Q	$\emptyset \leftarrow FALLO$	$NT = \{R\}$

Paso 9: $D(Q) = \emptyset \Rightarrow$ AC-3 devuelve INCONSISTENTE inmediatamente.

Estado final de los dominios:

Región	WA	NT	SA	Q	NSW	V	T
$D(\cdot)$	$\{G\}$	$\{R\}$	$\{B\}$	$\emptyset$	$\{G\}$	$\{R\}$	$\{R, G, B\}$

7.7.5 Conclusión

La asignación $\{WA = G,\ V = R\}$ es **inconsistente**. AC-3 lo demostró en 9 eliminaciones de 27 arcos procesados sin ningún backtracking. La cadena de propagación fue:

$$WA = G \ \rightarrow\ SA \text{ pierde } G \ \rightarrow\ SA = B \ \rightarrow\ NT \text{ pierde } B \ \rightarrow\ NT = R$$
$$V = R \ \rightarrow\ NSW \text{ pierde } R \ \rightarrow\ NSW = G \ \rightarrow\ Q \text{ pierde } G$$
$$NT = R \ \rightarrow\ Q \text{ pierde } R \ \rightarrow\ D(Q) = \emptyset$$

SA actúa como cuello de botella: con grado 5, su dominio se reduce a $\{B\}$ y propaga restricciones contradictorias que convergen en Q, vaciando su dominio.

References

[1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[2] Romero de la Cruz, A. (2026). *Presentaciones del curso de Inteligencia Artificial* [Material de clase]. Ingeniería en Computación, Universidad Nacional Autónoma de México (UNAM).